Advanced Database Technology

Compiled Revision Notes

Table of Contents

Week 1: Introduction to Database Systems and the Relational Model **4**
Three-Tier Web and Database Architecture 4
Evolution from File-Based Systems to DBMS 5
The Data Modelling Pyramid and Development Life Cycle 6
Case Study: The Data Modelling Lens (ABC Joinery Sales Office) 7
Case Study: Extracting a Data Model from a Real-World Narrative (Dentist Practice) . . 8
E. F. Codd's 1970 Paper: *A Relational Model of Data for Large Shared Data Banks* . . . 9
The Oracle / MySQL HR Schema 13

Week 2: Relational Algebra, Data Integrity, and Foundational SQL **15**
Formal Relational Terminology 15
Relational Algebra Operators 16
Relational Data Integrity and Keys 17
Peter Chen's 1976 Paper: *The Entity-Relationship Model—Toward a Unified View of Data* 18
Practical SQL Lab: The `EMP` and `DEPT` Schema (Worksheets 1, 2, and 3) 20

Week 3: Conceptual, Logical, and Physical Database Design and HR Schema SQL **24**
The Database Design Process 24
Entity-Relationship (ER) Model Constructs (Chen Notation) 25
Transforming (Mapping) an ER Model into a Relational Schema 28
Comprehensive SQL Lab: All 20 HR Schema Queries and Solutions 31

Week 4: DBMS Transaction Management and Concurrency Fundamentals **34**
Motivation for Transaction Management and Concurrency Control 34
Defining a Database Transaction (Tx) 35
Transaction Schedules: Serial, Serializable, and Non-Serializable 35
The ACID Properties of Transactions and Their MySQL Implementations 38
How `autocommit` Controls Transaction Boundaries in MySQL 39
Three Concurrency Control Techniques: Overview 39
Introduction to Pessimistic Locking and MySQL Storage Engines 40
How `InnoDB` Locks Rows via Index Files in Main Memory 42

Week 5: Advanced InnoDB Locking, Versioning, MVCC, and Practical Concurrency Lab **43**
Range Queries, Phantom Rows, Gap Locks, and Next-Key Locks 43
Specialized `InnoDB` Locks: Auto-Increment, Insert Intention, and Table Intention Locks . 45
Locking Pathologies: Deadlocks, Lost Updates, and Starvation 46
Optimistic Concurrency Control: Versioning and MVCC 47
Practical Concurrency Lab: `7. Locking.sql` and Live `InnoDB` Lock Monitor Analysis . . 49

Week 6: Codd's 12 Rules for Relational Database Management Systems and Data Quality **53**
Historical Context: Dr Edgar Frank "Ted" Codd and the 1985 Rules 53
Codd's Rule 0 and the 12 Rules in Detail 54

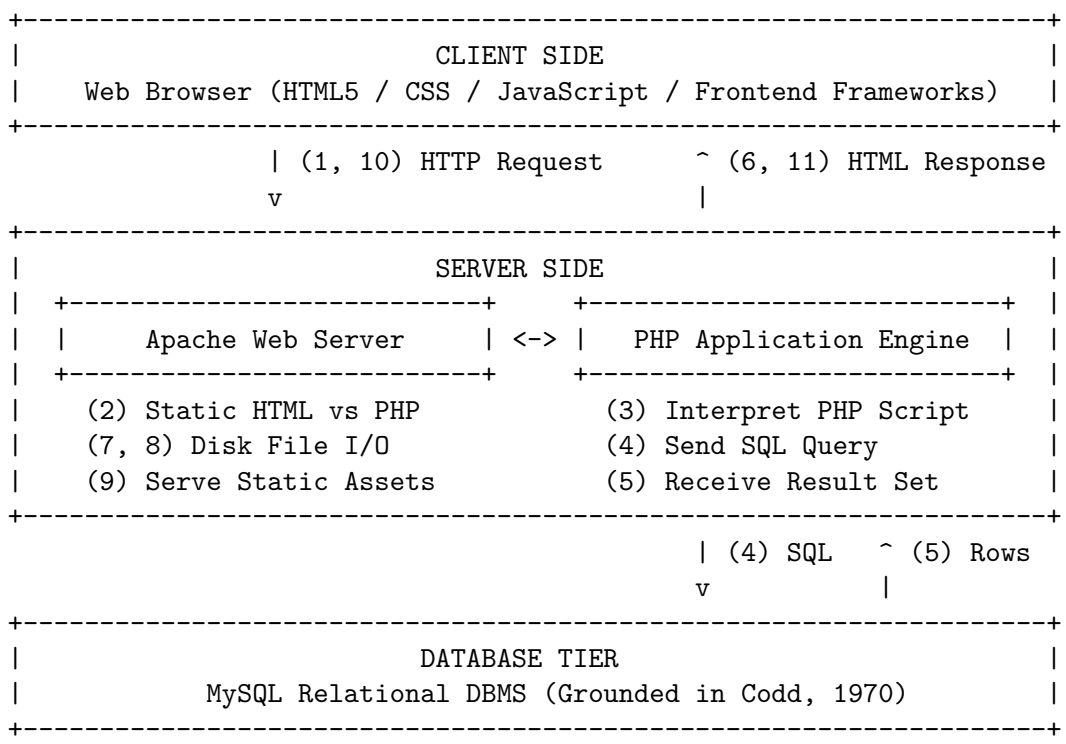
Week 7: Post-Relational (NoSQL) Databases, Data Architecture Patterns, and MongoDB **62**
Introduction to Post-Relational (NoSQL) Databases 62
The Four NoSQL Business Drivers: Why Single-Node RDBMSs Cracked 63
The Four NoSQL Data Store Families 65
Data Architecture Patterns: Row-Store RDBMS vs Key-Value Store Deep Dive 66
Distributed Database Theory: The CAP Theorem and ACID vs BASE 69
MongoDB Architecture, BSON, and CRUD Operations 71
Schema Design in MongoDB: Normalization (Referencing) vs Denormalization (Embedding) 74
Comprehensive Comparison Across Relational and NoSQL DBMS Families 75

Week 1: Introduction to Database Systems and the Relational Model

Three-Tier Web and Database Architecture

Overview of the Three-Tier Application Stack

Modern database-driven applications typically separate responsibilities across three distinct tiers: the **Client Side**, the **Server Side (Application Tier)**, and the **Database Tier**.



- **Client Side (Presentation Tier):**
  - Runs in the user's browser using HTML5, CSS, and JavaScript.
  - Client-side UI frameworks and libraries evolve rapidly, changing every few years to improve interactivity and presentation.
- **Server Side (Application / Middleware Tier):**
  - Hosts the web server (e.g., **Apache**) and server-side scripting engine (e.g., **PHP**).
  - Compared to client-side frameworks, the core request-processing model of Apache and PHP has remained relatively stable since around 1995.
- **Database Tier (Data Management Tier):**
  - Uses a Relational Database Management System (RDBMS) such as **MySQL** to persist, query, and protect structured data.
  - The relational tier rests on a rigorous mathematical foundation introduced by **Dr Edgar F. Codd (1970)** at IBM.

Step-by-Step Request Lifecycle Across the Stack

1. **Client Request (Step 1):** The browser issues an HTTP request for a dynamic page (e.g., `index.php`) to the Apache Web Server.

2. **Routing Dynamic vs Static Content (Step 2):** Apache inspects the requested resource. If it contains server-side code (`.php`), Apache forwards it to the PHP interpreter.
3. **Script Execution (Step 3):** The PHP engine parses and executes the application logic.
4. **Database Query Execution (Step 4):** When the script requires persistent data, PHP opens a connection to the **MySQL** server and transmits a structured SQL query.
5. **Result Set Return (Step 5):** MySQL parses, optimises, and executes the query against its stored relations and returns the matching rows to PHP.
6. **Dynamic HTML Generation (Step 6):** PHP formats the returned data into standard HTML and passes the rendered document back to Apache, which transmits it to the client browser.
7. **Static Asset Inspection (Steps 7–11):** As the browser parses the returned HTML, it discovers references to static assets (images, stylesheets, scripts) and sends follow-up requests (Step 10) to Apache. Apache retrieves those files directly from the server's local disk (Steps 7–8) and streams them back to the browser (Steps 9 and 11) without invoking PHP or MySQL.

Evolution from File-Based Systems to DBMS

Limitations of Pre-Database File Systems

Before Database Management Systems were developed, organisations stored data in flat files managed directly by individual application programs (e.g., COBOL or C programs reading sequential or indexed files).

- **Data Redundancy and Inconsistency:** Separate departments maintained their own copies of the same customer or employee records. Updating an address in one file while leaving another unchanged caused contradictory data across the organisation.
- **Program-Data Dependence:** The physical structure and record layout of every file was hard-coded into the application source code. Adding a single new field to a record required modifying and recompiling every program that read that file.
- **Lack of Centralised Integrity Enforcement:** Business rules (such as valid salary ranges or mandatory fields) had to be re-implemented inside every application program rather than declared once centrally.
- **Poor Concurrency and Crash Recovery:** Concurrent file writes by multiple users easily corrupted records, and a system crash mid-write left files in a half-updated, inconsistent state.
- **Inflexible Ad-Hoc Querying:** Answering a new business question required writing a brand-new procedural program to scan and merge files rather than issuing a declarative query.

The ANSI/SPARC Three-Level DBMS Architecture

To overcome program-data dependence, modern DBMS architectures separate how data is viewed by users, how it is logically structured, and how it is physically stored on disk:

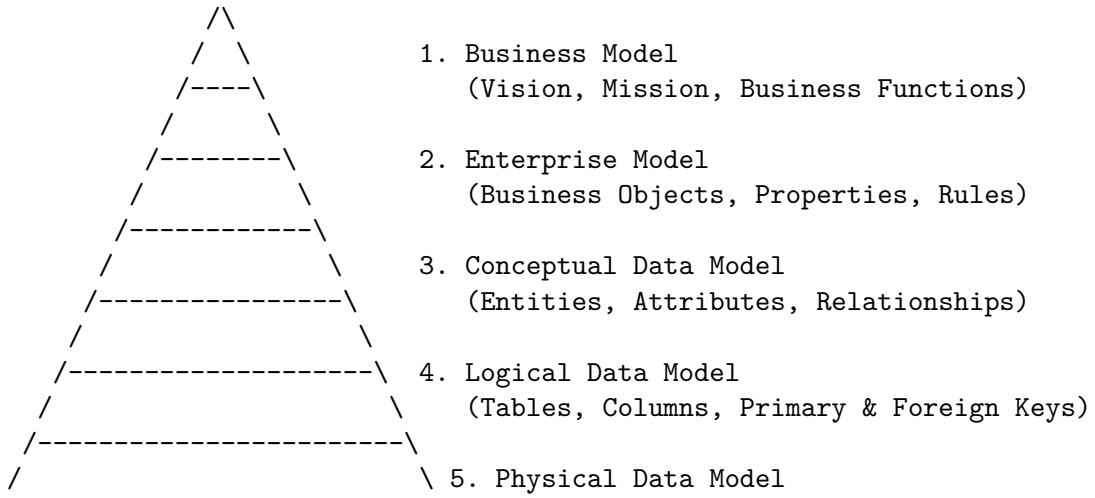
Architectural Level	Audience	Purpose and Description
External Level (View Level)	End Users & Application Programs	Provides tailored **Views** of the database relevant to specific users or roles while hiding irrelevant data and enforcing security restrictions.
Conceptual Level (Logical Level)	Database Designers & Administrators	Describes *what* data is stored in the entire database and the relationships, data types, keys, and constraints among them, completely independent of physical storage details.
Internal Level (Physical Level)	DBMS Storage Engine & OS	Describes *how* data is physically stored on hardware (file pages, B-tree/hash indexes, compression, block layouts, buffer pools, and disk pointers).

- **Logical Data Independence:** The ability to modify the **Conceptual Schema** (e.g., adding new tables or columns, or splitting a table while preserving a view) without breaking existing external views or application programs.
- **Physical Data Independence:** The ability to modify the **Internal Schema** (e.g., creating a new index, moving data files to a different disk, or changing page storage formats) without altering the conceptual schema or rewriting application queries.

The Data Modelling Pyramid and Development Life Cycle

Top-Down Design vs Bottom-Up Reverse Engineering

Database engineering bridges high-level business strategy and low-level physical storage through a five-layer **Data Modelling Pyramid**:



/_____\ (DDL Tables, CRUD Operations, Indexes, Disk)

- **Top-Down Approach (Development Life Cycle — From Abstract to Detail):**
  1. **Business Model:** Captures how the organisation operates in the real world (e.g., sales processing, customer billing, stock control).
  2. **Enterprise Model:** Identifies the core business **Objects** and their high-level **Properties** across the enterprise.
  3. **Conceptual Data Model:** Formalises those objects into **Entities**, **Attributes**, and **Relationships** (typically drawn as an Entity-Relationship Diagram), independent of any specific database technology.
  4. **Logical Data Model:** Translates the conceptual model into a technology-class structure—for a relational database, this consists of normalised **Tables (Relations)**, **Columns**, **Primary Keys (PKs)**, and **Foreign Keys (FKs)**.
  5. **Physical Data Model:** Implements the logical design inside a specific DBMS product (e.g., MySQL, Oracle, or MongoDB), defining SQL DDL statements, exact column data types, storage engines, indexes, partitioning, and **CRUD (Create, Read, Update, Delete)** operations.
- **Bottom-Up Approach (Reverse Engineering):**
  - Starts from an existing legacy database, flat files, or paper forms/spreadsheets at the **Physical / Logical** level and abstracts upwards to reconstruct the underlying **Conceptual** and **Business** models.

Case Study: The Data Modelling Lens (ABC Joinery Sales Office)

Problem Narrative and Context Diagram

Consider the **Sales Office at ABC Joinery**, which processes joinery orders from building contractors:

- **External Entities Interacting with the System:**
  1. **Builder (Customer):** Sends an *Order* into the Sales Office and receives a priced *Copy of Order* (confirmation/invoice).
  2. **Workshop:** Receives a *Copy of Order* detailing the items to manufacture and returns *Updated Order Details* once manufacturing progresses.
  3. **Accounts Department:** Sends *Customer Accounts Details* (credit status) to the Sales Office and receives a *Copy of Order* for invoicing.
  4. **Sales Manager:** Receives aggregated *Sales Reports* generated by the Sales Office.

Translating the Narrative Through the Modelling Layers

1. **Conceptual Model (Entities, Attributes, and Relationships):**
   - **Customer (`Cust`):** `CustNo` (Identifier), `CName`, `CAddr`, `CPhone`, `CredStat`.
   - **Order (`Ord`):** `OrdNo` (Identifier), `ODate`, `OValue`, `OStat`.
   - **Product (`Prod`):** `ProdCode` (Identifier), `PDesc`, `PPrice`, `StockLevel`.
   - **Relationships:** A `Customer` *places* one or more `Orders` (`1:M`), and an `Order` *contains* one or more `Products` (`M:N` via order line items recording quantity ordered).
2. **Relational Physical Structure vs Document Store Physical Structure:**
   - **Relational Physical Model (Normalized Tables):**

- CUSTOMER(CustNo PK, CName, CAddr, CPhone, CredStat)
- ORDERS(OrdNo PK, ODate, OValue, OStat, CustNo FK)
- PRODUCT(ProdCode PK, PDesc, PPrice, StockLevel)
- ORDER_LINE(OrdNo FK, ProdCode FK, QtyOrdered) — composite primary key (OrdNo, ProdCode) resolving the many-to-many relationship.

- **Document Store Physical Model (Embedded Hierarchy):**
  - A single self-contained Order JSON/BSON document embeds the customer summary and an array of line-item sub-documents (items: [{ prodCode, pDesc, qty, unitPrice }]), allowing the entire order to be retrieved or written in one atomic operation without multi-table joins.

3. **CRUD Operations Mapped to SQL:**
   - **Create:** INSERT INTO ORDERS VALUES (...)
   - **Read:** SELECT * FROM ORDERS JOIN CUSTOMER USING (CustNo) WHERE ...
   - **Update:** UPDATE ORDERS SET OStat = 'Dispatched' WHERE OrdNo = 105
   - **Delete:** DELETE FROM ORDERS WHERE OrdNo = 105

Case Study: Extracting a Data Model from a Real-World Narrative (Dentist Practice)

Narrative Analysis (Dr Mary Mulcahy's Dental Surgery)

In **Dr Mary Mulcahy's Dental Practice** (assisted by her secretary Helen), analysing the daily workflow reveals how raw domain descriptions map to database constructs:

- **Patient Registration & Charts:**
  - When a new patient phones in, Helen allocates the earliest available slot in the **Appointments Diary** (recording AppointmentDate, AppointmentTime, PatientName, and Phone).
  - Upon arrival, the patient fills out a registration form used to create a **Patient Chart** containing ChartNo (unique identifier), Name, Address, PhoneNumber, DateOfBirth, and Occupation.
- **Appointments, Visits, and Treatments:**
  - Every dental examination or clinical procedure performed during an appointment is recorded on the patient's chart with the VisitDate and TreatmentNotes.
  - A standard **Treatment Fee Schedule** provides guideline prices (TreatmentCode, Description, GuidelineFee), though the actual fee charged for a specific patient treatment may vary depending on clinical complexity.
- **Billing and Partial Payments:**
  - Helen generates **Bills** for treatments completed and mails them to patients (BillNo, BillDate, TotalAmount, ChartNo).
  - Patients may pay a bill in full or in **multiple partial instalments** (PaymentNo, PaymentDate, AmountPaid, BillNo), requiring a 1:M relationship between Bill and Payment rather than a single Paid (Y/N) column.
- **Cancellations and Specialist Referrals:**
  - Appointments may be cancelled or rearranged; late cancellations may incur a cancellation fee.
  - If specialist dental work is required, Dr Mulcahy sends a **Referral** to an external **Spe-**

cialist Consultant (`SpecialistID`, `SpecialistName`, `ClinicAddress`, `Specialty`), who subsequently returns a specialist report recorded against the patient's chart.

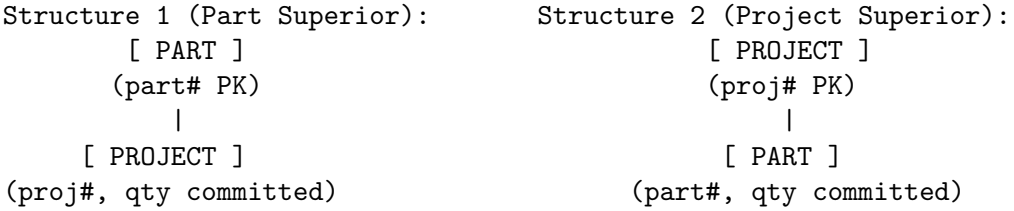
E. F. Codd's 1970 Paper: *A Relational Model of Data for Large Shared Data Banks*

Motivation: Failures of Pre-Relational (Hierarchical and Network) Systems

Prior to 1970, large commercial databases used **Tree-structured (Hierarchical)** models (such as IBM's IMS) or **Network** models (such as CODASYL / Bachman's IDS). Codd demonstrated that these systems forced application programmers to navigate low-level physical pointers, chains, and access paths. If the physical database structure was tuned or reorganised, thousands of application programs broke.

Codd identified **Three Data Dependencies** that must be eliminated to achieve true **Data Independence**:

1. **Ordering Dependence:**
   - Pre-relational systems often stored records in a specific physical sort order on disk (e.g., parts sorted by `part#`) and allowed application programs to rely on that physical ordering when iterating through records.
   - Because hardware-level ordering is closely tied to performance tuning, a DBMS must be free to change physical storage order at any time without breaking queries.
2. **Indexing Dependence:**
   - Performance-oriented indices increase query speed at the cost of slowing down updates and insertions.
   - If application programs explicitly reference named indices or index chains to locate data, the DBA cannot add, drop, or reorganise indices without breaking application code. Indices must remain purely transparent physical access aids.
3. **Access Path Dependence:**
   - Tree and network models force the schema designer to hard-code directed parent-child pointer hierarchies.
   - **Codd's 5-Tree Example (Parts, Projects, and Commitments):** Suppose a database tracks `Part` (`part#`, `part name`, `part description`, `quantity- on- hand`, `quantity- on- order`), `Project` (`proj#`, `proj name`, `proj description`), and the quantity of each part committed to a given project (`quantity committed`). Even for this tiny domain, there are **five distinct hierarchical tree structures** (`Structure 1` with `Part` superior to `Project`, `Structure 2` with `Project` superior to `Part`, `Structure 3` with two separate trees cross-linked via `Part-Project`, `Structure 4`, and `Structure 5`):



   - In a hierarchical DBMS, a query written for `Structure 1` ("print the part number and

quantity-on-hand for every part used in project P_2") fails if the database is reorganised into `Structure 2`.

- **The Connection Trap:** Network and hierarchical users frequently fell into the *connection trap*—assuming that because a path of pointers exists from entity A to entity B and from entity B to entity C, a meaningful semantic relationship automatically holds between A and C (which fails whenever the intermediate relation is non-functional or many-to-many).

Mathematical Foundation of the Relational View

Sets, Domains, and n-ary Relations Given sets (domains) $S_1, S_2, \ldots, S_n$ (not necessarily distinct), R is a **relation** on these n sets if it is a subset of the Cartesian product of the domains:

$$R \subseteq S_1 \times S_2 \times \cdots \times S_n$$

- **Degree (Arity):** R is said to be a relation of **degree** n ($n = 1$ is unary, $n = 2$ is binary, $n = 3$ is ternary, and n is n-ary).
- **Domain vs Active Domain:** The j-th domain S_j is the complete pool of permissible values for the j-th column, whereas the **active domain** at any instant is the subset of values from S_j that are actually present in relation R at that moment.

Five Properties of an Array (Table) Representation of a Relation When an n-ary relation R is represented as a two-dimensional rectangular array (table), it must satisfy five strict mathematical properties:

1. **Each row represents an n-tuple** of the relation R.
2. **The ordering of rows is immaterial** (a relation is a mathematical set of tuples, so swapping two rows does not change the relation).
3. **All rows are distinct** (no duplicate tuples may exist in a mathematical relation).
4. **The ordering of columns is significant** in a positional relation—the j-th column corresponds to domain S_j—*unless* each column is explicitly labelled with a unique domain/role name.
5. **The significance of each column is partially conveyed by labelling it** with the name of the corresponding domain.

Repeating Domains, Role Names, and Domain-Unordered Relationships A critical subtlety arises when **two or more columns of the same relation draw values from the identical underlying domain**.

- **Example (`component` bill-of-materials relation):** In a manufacturing assembly hierarchy, a part may contain other sub-parts in specific quantities. Both the containing assembly and the contained sub-part come from the same domain `part`:

```
component (sub.part, super.part, quantity)
```

- Simply labelling two columns `part` and `part` is ambiguous. By prefixing the domain name with a distinct **Role Name** (`sub.` and `super.`), the meaning of each column becomes self-describing.
- This allows Codd to distinguish between:

- A **Relation** (domain-ordered n-tuples where position matters), and
- A **Relationship** (a domain-unordered set of role-qualified values where column sequence no longer matters to the user).

Keys, Nonsimple Domains, and Normalization (First Normal Form)

Primary Keys and Foreign Keys

- **Primary Key (PK):** Usually one domain (or a minimal combination of domains, known as a **composite key**) of a given relation has values that uniquely identify each element (n-tuple) of that relation and is **non-redundant** (no column can be dropped from a composite primary key without losing uniqueness).
- **Foreign Key (FK):** A domain (or combination of domains) of relation R is a **foreign key** if it is not the primary key of R, but its elements are values drawn from the primary key of some relation S (where S and R may even be the same relation in a self-referencing relationship).

Simple vs Nonsimple Domains and the Normalization Process (1NF) In complex operational systems, a domain of a relation might itself be another relation (a **nonsimple domain** or repeating group). Codd illustrated this with an `employee` relation containing nested sub-relations for `jobhistory` (which itself nests `salaryhistory`) and `children`:

```
Unnormalized Hierarchical Relation Tree:

          employee (man#, name, birthdate, jobhistory, children)
                                                 /            \
                                                /              \
     jobhistory (jobdate, title, salaryhistory)      children (childname,
      ↪  birthyear)
                                    |
                    salaryhistory (salarydate, salary)
```

- **Eliminating Nonsimple Domains (Normalization to 1NF):**
  - Starting at the root relation (`employee`), the designer takes its **primary key (`man#`)** and propagates it downwards to expand each immediate subordinate relation's primary key, stripping the nested relation column out of the parent.
  - Repeating this top-down down the tree flattens the nested structure into four independent, flat **First Normal Form (1NF)** relations built strictly from **simple (atomic) domains**:

```
employee'      (man#, name, birthdate)
                ^^^^
jobhistory'    (man#, jobdate, title)
                ^^^^^^^^^^^^
salaryhistory' (man#, jobdate, salarydate, salary)
                ^^^^^^^^^^^^^^^^^^^^^^^^^
children'      (man#, childname, birthyear)
                ^^^^^^^^^^^^^^^
```

- Notice how the composite primary key expands at each level (`man#` → `(man#, jobdate)` → `(man#, jobdate, salarydate)`), completely eliminating pointers, repeating groups, and variable-length nested arrays!

Linguistic Aspects and Relational Operations in Codd (1970)

Universal Data Sublanguage and Symmetric Exploitation

- **Predicate Calculus Foundation:** Codd proposed that data should be queried declaratively using a universal data sublanguage grounded in **first-order predicate calculus** (specifying *what* data is wanted via logical predicates rather than *how* to navigate pointers).
- **Symmetric Exploitation:** Unlike tree or network systems where queries depend on which record is at the top of the hierarchy, a relational system treats all columns of a relation symmetrically—any combination of attributes can serve as search predicates or output targets.
- **Named Set vs Expressible Set:**
  - The **Named Set** is the collection of fundamental base relations explicitly defined in the schema and maintained by the DBMS.
  - The **Expressible Set** is the much larger family of relations that can be derived dynamically from the named set through relational expressions and views.

Core Operations on Relations Defined by Codd

1. **Permutation:** Interchanging the column ordering of an array representation of a relation.
2. **Projection (π):** Selecting a specified subset of columns from relation R and automatically **eliminating any duplicate rows** that result:

$$\pi_L(R)$$

where L is the list of projected column indices or role names. 3. **Natural Join ($R * S$):** Combining two relations $R(A, B)$ and $S(B, C)$ that share a common domain B into a unified relation (A, B, C) wherever the B-values match, while preserving every tuple's join capability without loss of information. 4. **Tie (γ) and Cyclic Joins:** Joining relations where the final column of a chain is constrained to equal the initial column (e.g., turning a linear 3-join $R(A, B) * S(B, C) * T(C, A)$ into a cyclic 3-join where both A endpoints must match). 5. **Composition ($R \cdot S$):** Forming the natural join of $R(A, B)$ and $S(B, C)$ and immediately projecting out the connecting middle domain B:

$$R \cdot S = \pi_{13}(R * S)$$

6. **Restriction ($R_{L|M}S$):** Filtering a relation R using the values present in another relation S along matching domain lists L and M.

Redundancy and Consistency in Relational Models

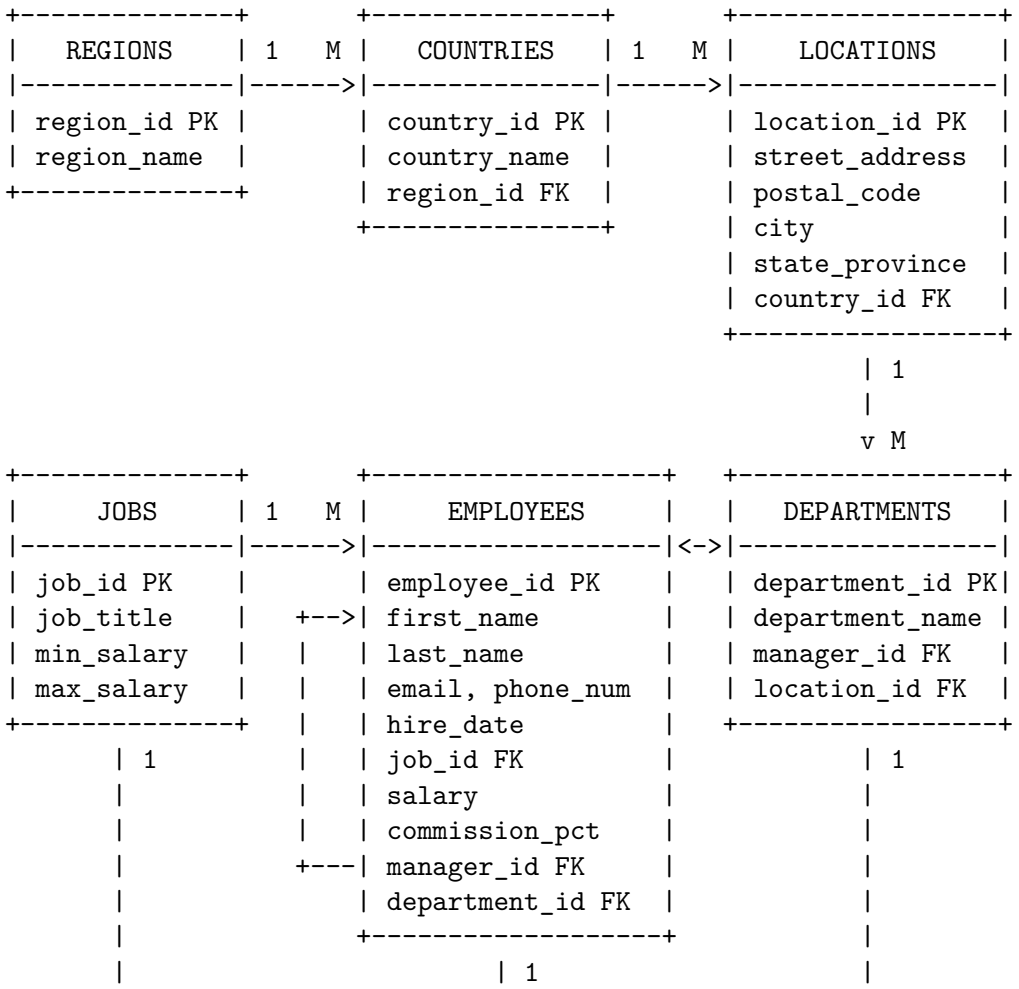
To ensure that a collection of relations remains free of update anomalies while still permitting convenient derived views, Codd distinguished between two forms of redundancy:

- **Strong Redundancy:** A set of relations is *strongly redundant* if it contains at least one relation that has a projection (potentially all columns) derivable from other relations in the collection **at all times** (e.g., storing a permanent column that equals the join/projection of two other base tables).
- **Weak Redundancy:** A set of relations is *weakly redundant* if it contains a relation that has a projection which is *not* derivable from other relations at all times, **but is at all times a projection of some natural join** of other relations in the collection (for instance, when two tables share common values that could lose unreferenced keys if only the join were stored).
- **Consistency of a Data Bank State** (C, Z, V)**:** Given a named set of relations C, a set of time-independent integrity constraints Z, and an instantaneous set of populate values V, the database state (C, Z, V) is **consistent** if and only if all values in V satisfy every constraint in Z.

The Oracle / MySQL HR Schema

Overview of the 7 HR Schema Tables

Throughout the module labs and SQL exercises, two classic relational schemas are used: the 2-table `EMP/DEPT` schema and the 7-table **Human Resources (`HR`) Schema**:



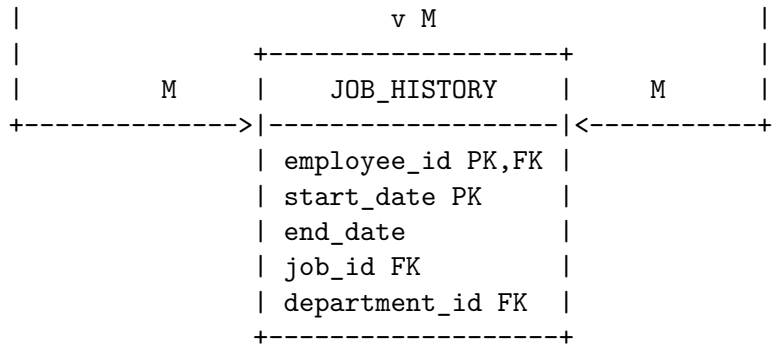


Table Name	Primary Key (PK)	Foreign Keys (FK)	Structural Role and Notes
REGIONS	region_id	*None*	Top-level geographic lookup (e.g., Europe, Americas, Asia).
COUNTRIES	country_id (CHAR(2))	region_id → REGIONS	Maps ISO country codes to a geographic region (M:1).
LOCATIONS	location_id	country_id → COUNTRIES	Physical office street addresses and cities (M:1 to COUNTRIES).
DEPARTMENTS	department_id	location_id → LOCATIONSmanager_id → EMPLOYEES	Organisational departments; participates in a bidirectional relationship with EMPLOYEES.
JOBS	job_id	*None*	Job role catalogue defining job_title, min_salary, and max_salary.
EMPLOYEES	employee_id	job_id → JOBSdepartment_id → DEPARTMENTSmanager_id → EMPLOYEES	Core staff table; includes a **unary (recursive) self-join** (manager_id references employee_id).
JOB_HISTORY	(employee_id, start_date)	employee_id → EMPLOYEESjob_id → JOBSdepartment_id → DEPARTMENTS	Historical audit table with a **composite primary key** tracking past roles held by employees.

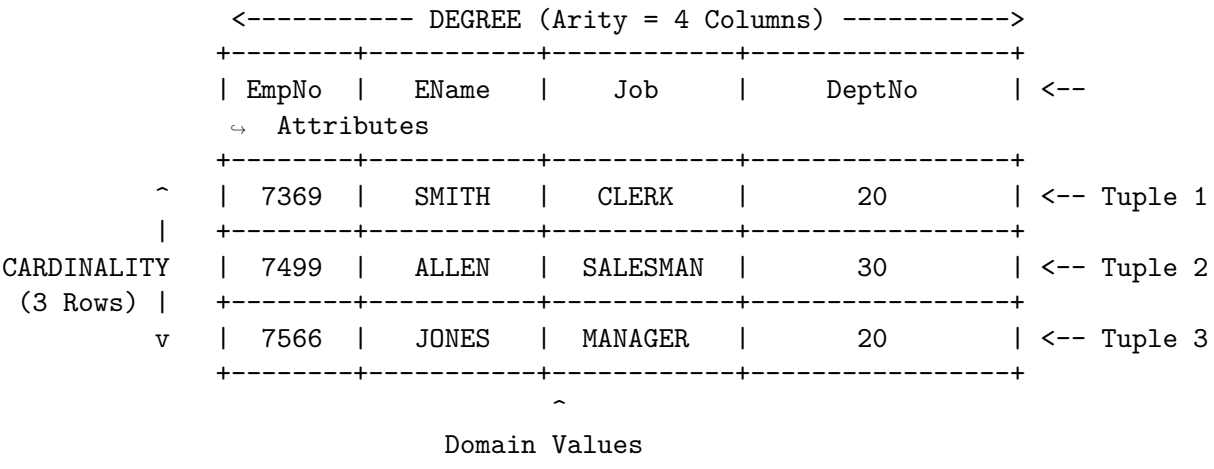
Week 2: Relational Algebra, Data Integrity, and Foundational SQL

Formal Relational Terminology

Relations, Tuples, Attributes, Degree, and Cardinality

In the relational model, informal SQL table terms correspond to precise mathematical concepts:

Formal Relational Term	Informal SQL Equivalent	Mathematical Definition
Relation	Table	A named two-dimensional set of n-tuples representing an entity type or relationship.
Tuple	Row / Record	A single ordered horizontal entry in a relation representing one real-world instance.
Attribute	Column / Field	A named vertical category of data drawn from an underlying domain.
Domain	Data Type + Constraints	The pool of permissible atomic values from which one or more attributes draw their values.
Degree (Arity)	Number of Columns	The number of attributes in the relation schema (relatively static over time).
Cardinality	Number of Rows	The number of tuples currently stored in the relation (changes frequently as rows are inserted/deleted).



Domains and SQL `CREATE DOMAIN`

A **Domain** defines the legal data type, length, format, and value constraints for an attribute (e.g., a `Names` domain of `VARCHAR(20)` or a `DeptCode` domain restricted to integers between `10` and `99`). Declaring domains explicitly prevents nonsense comparisons (such as joining an Employee Age column with a Department Number column simply because both happen to be stored as integers):

```sql
CREATE DOMAIN Names AS VARCHAR(20) NOT NULL;
CREATE DOMAIN SalaryRange AS DECIMAL(8,2)
  CHECK (VALUE >= 0 AND VALUE <= 250000);
```

Relational Algebra Operators

Relational algebra is a procedural query language where every operator accepts one or two relations as input and produces a **new relation** as output (the **closure property**). Codd divided relational algebra into **Set-Theoretic Operators** and **Relation-Specific Operators**.

Type Compatibility (Union Compatibility)

Three of the four set-theoretic operators (**Union**, **Intersection**, and **Difference**) require their two input relations R_1 and R_2 to be **Type Compatible (Union Compatible)**:

1. **Same Degree:** R_1 and R_2 must have the exact same number of attributes ($n = m$).
2. **Compatible Domains:** Each corresponding pair of attributes $\text{dom}(A_i)$ and $\text{dom}(B_i)$ ($1 \leq i \leq n$) must be drawn from the same (or compatible) underlying domain.

Note: Cartesian Product ($R_1 \times R_2$) does **not** require type compatibility!

Set-Theoretic Operators

Suppose we have two type-compatible relations R_1 and R_2:

```
Relation R1:                    Relation R2:
+-------+---------+             +-------+---------+
| EmpID |  EName  |             | EmpID |  EName  |
+-------+---------+             +-------+---------+
|  E1   |  Smith  |             |  E2   |  Jones  |
|  E2   |  Jones  |             |  E3   |  Blake  |
+-------+---------+             +-------+---------+
```

1. **Union ($R_1 \cup R_2$):**
   - **Definition:** Builds a relation containing all tuples that appear in R_1, or in R_2, or in both, with **duplicate tuples automatically eliminated**.
   - **Requirement:** R_1 and R_2 must be **Type Compatible**.
   - **Result of $R_1 \cup R_2$:** `{(E1, Smith), (E2, Jones), (E3, Blake)}` (3 tuples).
2. **Intersection ($R_1 \cap R_2$):**
   - **Definition:** Builds a relation containing only those tuples that appear in **both** R_1 and R_2.
   - **Requirement:** R_1 and R_2 must be **Type Compatible**.
   - **Result of $R_1 \cap R_2$:** `{(E2, Jones)}` (1 tuple).
3. **Difference ($R_1 - R_2$):**
   - **Definition:** Builds a relation containing all tuples that appear in R_1 **but do not appear** in R_2.

- **Order Matters (Non-Commutative):** $R_1 - R_2 \neq R_2 - R_1$.
- **Requirement:** R_1 and R_2 must be **Type Compatible**.
- **Result of $R_1 - R_2$:** `{(E1, Smith)}` (whereas $R_2 - R_1$ yields `{(E3, Blake)}`).

4. **Cartesian Product ($R_1 \times R_2$):**
   - **Definition:** Concatenates every tuple of R_1 with every tuple of R_2.
   - **Requirement:** Type compatibility is **not** required.
   - **Degree and Cardinality of $R_1 \times R_2$:**
     - $\text{Degree}(R_1 \times R_2) = \text{Degree}(R_1) + \text{Degree}(R_2)$
     - $\text{Cardinality}(R_1 \times R_2) = \text{Cardinality}(R_1) \times \text{Cardinality}(R_2)$

Relation-Specific Operators

1. **Restriction / Selection ($\sigma_{\textbf{predicate}}(R)$):**
   - **Horizontal slicing** of a relation: extracts the subset of **rows (tuples)** from R that satisfy a boolean condition (e.g., $\sigma_{\text{DEPTNO=30}}(\text{EMP})$).
   - Degree remains unchanged; cardinality is $\leq \text{Cardinality}(R)$.
2. **Projection ($\pi_{A_1, A_2, \ldots, A_k}(R)$):**
   - **Vertical slicing** of a relation: extracts the specified **columns (attributes)** from R and removes any duplicate rows from the result.
3. **Join ($R_1 \bowtie_{\textbf{condition}} R_2$):**
   - Combines related tuples from two relations into single longer tuples where a join condition holds between their attributes (equivalent to a Cartesian product followed by a Selection: $\sigma_{\text{condition}}(R_1 \times R_2)$).
   - In a **Natural Join ($R_1 \bowtie R_2$)**, equality is tested across all identically named common attributes and duplicate join columns are projected out.
4. **Division ($R_1 \div R_2$):**
   - Used for **"for all"** universal quantification queries (e.g., *"Find all suppliers who supply **every** part in project P_1"*).
   - If $R_1(A, B)$ and $R_2(B)$, then $R_1 \div R_2$ returns the values of A in R_1 that are paired with **every** B-value present in R_2.

Relational Data Integrity and Keys

Superkeys, Candidate Keys, Primary Keys, and Foreign Keys

- **Superkey:** Any attribute or set of attributes that uniquely identifies a tuple within a relation (may contain extra, unnecessary attributes).
- **Candidate Key:** A **minimal superkey**—an attribute or composite set of attributes that uniquely identifies every tuple in the relation, and from which no attribute can be removed without destroying uniqueness.
  - *Example:* In a `Vehicle_Owner` table, `Registration_No` is a simple candidate key, while the composite combination `(Name, DOB)` may serve as a second candidate key.
- **Primary Key (PK):** The candidate key chosen by the database designer to serve as the principal unique identifier for tuples in the relation. Every attribute participating in a primary key is strictly **prohibited from being `NULL` (Entity Integrity)**.

- **Alternate Key:** Any candidate key that was *not* chosen to be the primary key (enforced in SQL via `UNIQUE NOT NULL`).
- **Foreign Key (FK):** An attribute (or composite group of attributes) in a child relation whose values are required to match the primary key values of some tuple in a parent relation (or be wholly `NULL` if participation is optional) (**Referential Integrity**).

Declaring Composite Keys and Referential Integrity in SQL DDL

```
CREATE TABLE Student (
    Name        VARCHAR(30) NOT NULL,
    DOB         DATE        NOT NULL,
    Address     VARCHAR(60),
    Tutor_Name  VARCHAR(30),
    PRIMARY KEY (Name, DOB),
    FOREIGN KEY (Tutor_Name) REFERENCES Tutor(Tutor_Name)
        ON DELETE SET NULL
        ON UPDATE CASCADE
);
```

Peter Chen's 1976 Paper: *The Entity-Relationship Model—Toward a Unified View of Data*

Bridging Semantic Real-World Meaning and Logical Data Structures

While Codd's 1970 relational model freed databases from physical pointers, **Peter Pin-Shan Chen (1976)** observed that designers still needed a unified, top-down **semantic modelling methodology** to capture how humans naturally perceive the real world (in terms of **entities** and **relationships**) before mapping those structures into relational tables, network sets, or hierarchical trees.

Chen's Four Levels of Logical Views of Data

Level	Name	Core Constructs & Representation	Characteristics
Level 1	**Information Concerning Entities and Relationships**	**Entity** (e), **Relationship** (r), **Role**, **Entity Set** (E_i), **Relationship Set** (R_i), **Value Set** (V_i / Domain), and **Attribute** ($f : E_i$ or $R_i \to V_i$).	Pure conceptual model existing in human minds; completely independent of storage representation.

Level	Name	Core Constructs & Representation	Characteristics
Level 2	**Information Structure**	Regular **Entity Relations**, Weak Entity Relations, Regular **Relationship Relations**, and Weak Relationship Relations.	Organises Level 1 conceptual tokens into structured tuple tables separating entity relations from relationship relations.
Level 3	**Access-Path-Independent Data Structure**	Codd's **3NF Relational Tables**.	Merges/transforms Level 2 structures into access-path-independent relational tables manipulated via relational algebra/calculus.
Level 4	**Access-Path-Dependent Data Structure**	Network (CODASYL DBTG records and owner-member sets) or Hierarchical trees.	Commits to explicit physical access paths and pointer chains for high-speed navigation where required.

Formal Mathematical Definitions in Chen's ER Model

1. **Entity and Entity Set (E_i):** An *entity* is a "thing" which can be distinctly identified (e.g., a specific person, company, or event). Entities are classified into different **entity sets** $E_1, E_2, \ldots, E_n$ (such as `EMPLOYEE`, `PROJECT`, `DEPARTMENT`).
2. **Relationship and Relationship Set (R_i):** A *relationship* is an association among entities. A mathematical **relationship set** R_i among n entity sets $E_1, E_2, \ldots, E_n$ is a set of relationship tuples:

$$R_i = \{[e_1, e_2, \ldots, e_n] \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

where each e_j plays a specific **role** r_j in the relationship. 3. **Value Sets (V_i) and Attributes as Functions (f):** - Unlike informal descriptions, Chen formally defines an **Attribute** not as a value itself, but as a **mathematical function** f that maps from an Entity Set E_i (or a Relationship Set R_i) into a Value Set V_i (or a Cartesian product of value sets for composite attributes):

$$f : E_i \text{ (or } R_i) \to V_j$$

- Crucially, **both entities AND relationships can have attributes** (for example, `PERCENTAGE-OF-TIME` or `START-DATE` is an attribute of the `PROJECT-WORKER` relationship rather than of `EMPLOYEE` or `PROJECT` alone).

4. **Top-Down ER Design vs Bottom-Up 3NF Synthesis:**

- Bottom-up 3NF synthesis starts from an unstructured pile of attributes and functional dependencies, which can miss subtle semantic distinctions (such as multiple relationships between the same two entities).
- Top-down ER modelling captures the semantic structure first (Entities → Relationships → Attributes) and then maps cleanly into **3NF relations**.

Practical SQL Lab: The EMP and DEPT Schema (Worksheets 1, 2, and 3)

The EMP and DEPT Schema Structure

```
CREATE TABLE DEPT (
    DEPTNO INT PRIMARY KEY,
    DNAME  VARCHAR(14),
    LOC    VARCHAR(13)
);

CREATE TABLE EMP (
    EMPNO    INT PRIMARY KEY,
    ENAME    VARCHAR(10),
    JOB      VARCHAR(9),
    MGR      INT,
    HIREDATE DATE,
    SAL      DECIMAL(7,2),
    COMM     DECIMAL(7,2),
    DEPTNO   INT,
    FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO),
    FOREIGN KEY (MGR)    REFERENCES EMP(EMPNO)
);
```

- **DEPT Rows:** `(10, 'ACCOUNTING', 'NEW YORK')`, `(20, 'RESEARCH', 'DALLAS')`, `(30, 'SALES', 'CHICAGO')`, `(40, 'OPERATIONS', 'BOSTON')`.
- **Self-Referencing Foreign Key (MGR):** `EMP.MGR` points back to `EMP.EMPNO` in the same table (`KING` has `MGR IS NULL` as President).

SQL Problem Sheet 1 (Worksheet 1): Filtering, Range, and Set Predicates

1. **Describe the structure of the EMP table:**

```
DESCRIBE EMP;
```

2. **List all of the information in all of the fields in the EMP table:**

```
SELECT *
FROM EMP;
```

3. **List the name and number of all the departments in the DEPT table:**

```
SELECT DNAME, DEPTNO
FROM DEPT;
```

4. **List all of the information in the `EMP` table for those employees who are in department number 30:**

```
SELECT *
FROM EMP
WHERE DEPTNO = 30;
```

5. **Retrieve all of the information in the `EMP` table from those employees who are `MANAGER`s and who earn more than 2500 per year:**

```
SELECT *
FROM EMP
WHERE JOB = 'MANAGER'
  AND SAL > 2500;
```

6. **List the name, job, and department number of all of those employees who are managers and who are not in department number 30 (`Hint: !=`):**

```
SELECT ENAME, JOB, DEPTNO
FROM EMP
WHERE JOB = 'MANAGER'
  AND DEPTNO != 30;
```

7. **List the name and salary of all the employees in the `EMP` table who earn between 1200 and 1400 (`Hint: BETWEEN A and B`):**

```
SELECT ENAME, SAL
FROM EMP
WHERE SAL BETWEEN 1200 AND 1400;
```

8. **List all of the information in `DEPT` about those departments whose number is either 10 or 30 (`Hint: IN (10,30)`):**

```
SELECT *
FROM DEPT
WHERE DEPTNO IN (10, 30);
```

9. **Bonus Class Query — List the names of all Managers and the names of their staff (Self-Join on `EMP`):**

```
SELECT m.ENAME AS Manager_Name,
       e.ENAME AS Staff_Name
FROM EMP e
JOIN EMP m ON e.MGR = m.EMPNO
ORDER BY m.ENAME, e.ENAME;
```

- *Why a Self-Join is Required:* Because both staff members and managers live in the single `EMP` table, we alias `EMP` twice—once as `e` (the subordinate whose `MGR` column holds a manager's ID) and once as `m` (the manager whose `EMPNO` matches `e.MGR`).

SQL Problem Sheet 2 (Worksheet 2): Sorting (`ORDER BY`), `DISTINCT`, and Handling `NULL` Commissions

1. **List the salaries, jobs, and names of the employees who are in department 30. Display the rows in salary order (`ASC` is default):**

```
SELECT SAL, JOB, ENAME
FROM EMP
WHERE DEPTNO = 30
ORDER BY SAL ASC;
```

2. **List the job, salary, and employee name of all those employees in the `EMP` table. Display the rows in job order and then in descending salary order (`Hint: ORDER BY, DESC`):**

```
SELECT JOB, SAL, ENAME
FROM EMP
ORDER BY JOB ASC,
         SAL DESC;
```

3. **Get a listing of the distinct jobs in the `EMP` table (`Hint: DISTINCT`):**

```
SELECT DISTINCT JOB
FROM EMP;
```

4. **Get the employee name and department number of all employees called `ADAMS`:**

```
SELECT ENAME, DEPTNO
FROM EMP
WHERE ENAME = 'ADAMS';
```

5. **Get the location of department number 20:**

```
SELECT LOC
FROM DEPT
WHERE DEPTNO = 20;
```

6. **Get the employee name and total salary of all employees who are salesmen (`Hint: Sal + Comm`):**

```
SELECT ENAME,
       SAL + IFNULL(COMM, 0) AS Total_Salary
FROM EMP
WHERE JOB = 'SALESMAN';
```

- *Critical `NULL` Trap:* If a query simply writes `SELECT ENAME, SAL + COMM FROM EMP WHERE JOB = 'SALESMAN'`, any salesman whose `COMM` is `NULL` would evaluate `SAL + NULL` to `NULL`! Wrapping `COMM` in `IFNULL(COMM, 0)` (or standard SQL `COALESCE(COMM, 0)`) safely treats missing commissions as `0`.

Worksheet 3: DML (`INSERT`, `UPDATE`, `DELETE`), Joins, and Grouping (`GROUP BY` / `HAVING`)

1. **Add a record containing a new department number (`50`), name (`'PENSIONS'`), and location (`'CHICAGO'`) into the `DEPT` table:**

```
INSERT INTO DEPT (DEPTNO, DNAME, LOC)
VALUES (50, 'PENSIONS', 'CHICAGO');
```

2. **Get the employee name and department location of the employee called `ADAMS`:**

```
SELECT e.ENAME, d.LOC
FROM EMP e
JOIN DEPT d ON e.DEPTNO = d.DEPTNO
WHERE e.ENAME = 'ADAMS';
```

3. **Get the maximum salary paid in each department:**

```
SELECT DEPTNO,
       MAX(SAL) AS Max_Salary
FROM EMP
GROUP BY DEPTNO;
```

4. **Update the `EMP` table to give all clerks a 100 increase in salary:**

```
UPDATE EMP
SET SAL = SAL + 100
WHERE JOB = 'CLERK';
```

5. **Get the employee name and total salary of all employees who are salesmen:**

```
SELECT ENAME,
       SAL + IFNULL(COMM, 0) AS Total_Salary
FROM EMP
WHERE JOB = 'SALESMAN';
```

6. **Delete all records in the `DEPT` table which have a department number of `50` and then view the contents of the table to ensure that the operation has been completed correctly:**

```
DELETE FROM DEPT
WHERE DEPTNO = 50;

SELECT *
FROM DEPT;
```

7. **Get the number of employees in each job in each department in the company. Also list the department they are in, the job title, the sum of the salaries of each employee type in each department, the average salary of each employee type in each department, and show the information grouped by department name and job (`Hint: Use SUM(SAL), COUNT(*), AVG(SAL) and GROUP BY`):**

```
SELECT d.DNAME,
       e.JOB,
       COUNT(*)              AS Num_Employees,
       SUM(e.SAL)            AS Total_Sal,
```

```
       ROUND(AVG(e.SAL), 2)  AS Avg_Sal
FROM EMP e
JOIN DEPT d ON e.DEPTNO = d.DEPTNO
GROUP BY d.DNAME, e.JOB
ORDER BY d.DNAME, e.JOB;
```

8. **Show the same information as query 7 does, but only show the groups having at least two employees (`Hint: HAVING COUNT(*)`):**

```
SELECT d.DNAME,
       e.JOB,
       COUNT(*)              AS Num_Employees,
       SUM(e.SAL)            AS Total_Sal,
       ROUND(AVG(e.SAL), 2)  AS Avg_Sal
FROM EMP e
JOIN DEPT d ON e.DEPTNO = d.DEPTNO
GROUP BY d.DNAME, e.JOB
HAVING COUNT(*) >= 2;
```

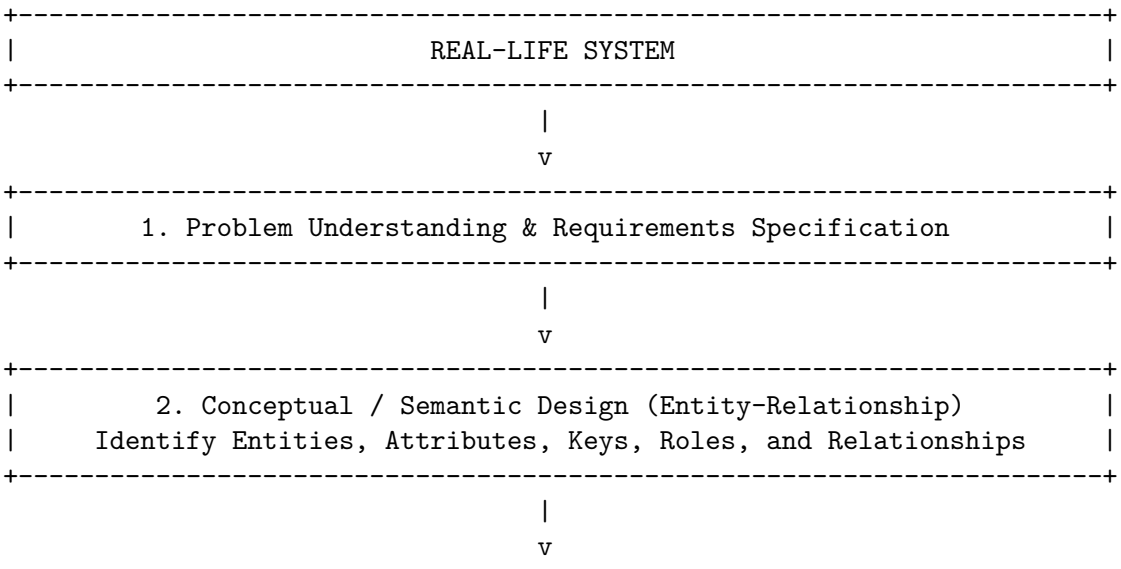
- *`WHERE` vs `HAVING`:* `WHERE` filters individual rows **before** grouping occurs (and cannot reference aggregate functions), whereas `HAVING` filters whole groups **after** `GROUP BY` aggregation has been computed.

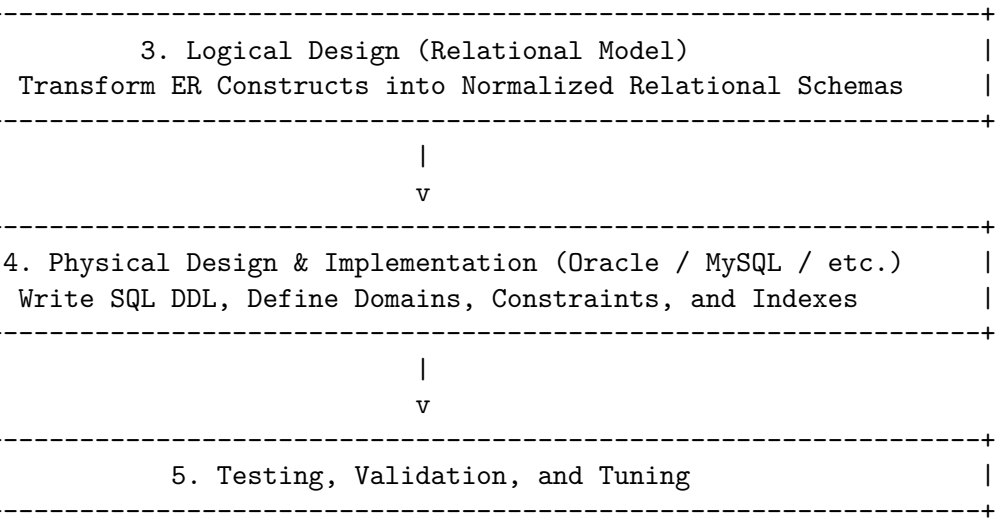
Week 3: Conceptual, Logical, and Physical Database Design and HR Schema SQL

The Database Design Process

Iterative Multi-Stage Engineering Pipeline

Designing a database from a real-life system follows a structured pipeline that moves from user-oriented semantics to DBMS-specific implementation:





- **Iterative Nature of Conceptual Design:**
  - In practice, database design is **iterative rather than strictly linear**.
  - A practical design heuristic proceeds through:
    1. Identify candidate **Entities** (typically nouns in the requirements narrative).
    2. Identify **Attributes** describing those entities.
    3. Analyse attribute uniqueness to refine which concepts are true entities versus attributes or relationships.
    4. Identify **Primary Key** attributes for each strong entity.
    5. Identify **Relationships** (typically verbs linking nouns), their **Cardinality**, and **Participation** constraints.
    6. Add **Role Names** (for recursive or multi-relationship links) and **Relationship Attributes**, iterating back and forth until the conceptual model accurately mirrors the domain.

Entity-Relationship (ER) Model Constructs (Chen Notation)

Visual Summary of Chen ER Diagram Symbols

ER Construct	Chen Diagram Symbol	Meaning and Example
Strong (Regular) Entity	Single-line Rectangle `[ Entity ]`	Exists independently of any other entity (e.g., `Student`, `Customer`, `Staff`).
Weak Entity	Double-line Rectangle `[[ Weak Entity ]]`	Existence-dependent on an identifying owner entity; cannot be uniquely identified by its own attributes alone (e.g., `Book-Loan`, `Account`).
Identifying Relationship	Double-line Diamond `<< Relationship >>`	Links a **Weak Entity** to its owner Strong Entity.
Regular Relationship	Single-line Diamond `< Relationship >`	Association linking one, two, or three entity types.

ER Construct	Chen Diagram Symbol	Meaning and Example
Simple Attribute	Single-line Oval `( Attribute )`	Atomic property that cannot be subdivided further (e.g., `sex`, `salary`).
Primary Key Attribute	Oval with **Underlined** Name `( _PK_ )`	Uniquely identifies each instance of a strong entity (e.g., `matric-no`).
Composite Attribute	Oval branching into smaller component Ovals	Attribute composed of structured sub-parts (e.g., `address` → `no`, `code`, `city`; `date-of-birth` → `year`, `month`, `day`).
Multi-Valued Attribute	**Thick / Double-line Oval** `(( tel-no ))`	An attribute that can hold multiple simultaneous values for a single entity instance (e.g., multiple phone numbers).
Derived Attribute	**Dashed Oval** `(: age :)`	Value computed dynamically from another stored attribute rather than physically stored (e.g., `age` derived from `date-of-birth`, or `fees`).
Total Participation	**Double Line** `===` connecting Entity to Diamond	**Mandatory participation:** *Every* instance of the entity must participate in at least one instance of the relationship.
Partial Participation	**Single Line** `---` connecting Entity to Diamond	**Optional participation:** An entity instance may exist without participating in the relationship.

Classes of Entities: Strong vs Weak Entities

- **Strong (Regular) Entity Type:**
  - Possesses its own primary key (`Customer(Name, DOB, Address, Job, Salary)` or `Student(matric-no, name, sex)`).
  - Its existence does not depend on any other entity in the schema.
- **Weak Entity Type:**
  - Does not possess sufficient attributes to form a globally unique primary key on its own; instead, it has only a **partial key (discriminator)** (such as `Type` on an `Account` or `Copy-No` on a `Book-Loan`).
  - Its existence depends on an **Identifying Owner Entity** via an **Identifying Relationship** (drawn with a double diamond). If the owner `Customer` is deleted, their dependent weak `Account` instances cease to exist.

Classification of Attributes (Properties)

1. **Simple vs Composite Attributes:**
   - **Simple (Atomic):** Cannot be broken down further (`sex`, `title`, `salary`).
   - **Composite:** Made up of more basic atomic components (`address` → `no`, `code`, `city`, `tel-no`; `date-of-birth` → `year`, `month`, `day`). When mapped to a relational table, only the **leaf-level simple components** become columns so 1NF atomicity is preserved.
2. **Single-Valued vs Multi-Valued Attributes:**
   - **Single-Valued:** Holds at most one value per entity instance (`matric-no`, `date-of-birth`).
   - **Multi-Valued:** Can hold a set of values for a single entity (e.g., `tel-no` drawn as a thick/double oval). In a relational database, a multi-valued attribute must be extracted into a separate table to satisfy 1NF.
3. **Base (Stored) vs Derived Attributes:**
   - **Base:** Physically stored in the database (`date-of-birth`).
   - **Derived:** Calculated at runtime from base attributes (`age` computed from `CURRENT_DATE - date-of-birth`, or `fees` computed from course credits), drawn with a dashed oval.

Degree of Relationships, Role Names, and Relationship Attributes

1. **Degree of a Relationship (Number of Participating Entity Types):**
   - **Binary Relationship (Degree 2):** Connects two entity types (e.g., `Student <belongs to> Department`). This is the most common relationship degree.
   - **Ternary Relationship (Degree 3):** Connects three entity types simultaneously around a single diamond:

```
[ Student ] ------- < is taught > ------- [ Tutor ]
                          |
                          |
                     [ Module ]
```

(Read as: A `Student` is taught a specific `Module` by a specific `Tutor`).

   - **Unary / Recursive Relationship (Degree 1):** Connects an entity type **to itself**, where instances of the same entity play two distinct **Roles**:

```
          representative
       +--------------------+
       |                    |
       v                    |
 [ Student ]        < represents >
       ^                    |
       |                    |
       +--------------------+
        (represented) group
```

2. **Multiple Relationships and Role Names:**
   - The same pair of entity types can be connected by **more than one distinct relationship**, each carrying explicit **Role Names** on the connecting lines:

- `Staff` *(role: head of department)* `<manages>` `Department` *(role: department).*
- `Staff` *(role: member of staff)* `<is affiliated to>` `Department` *(role: department).*

3. **Relationships with Attributes:**
   - Attributes that describe the interaction *between* entities rather than either entity in isolation must be attached directly to the **Relationship Diamond**.
   - *Example:* In `Staff <is affiliated to> Department`, the attributes `date`, `duration`, and `position` belong to the `<is affiliated to>` relationship because a staff member may be affiliated with multiple departments at different dates and in different positions.
   - *Relational vs Document Mapping Note:* In a relational model, an `M:N` relationship with attributes becomes a separate **junction/associative table**; in a NoSQL document store, it is typically modelled as an **embedded array of affiliation sub-documents** inside the `Staff` (or `Department`) document.

Constraints on Relationships: Participation and Cardinality

1. **Participation Constraints (Total vs Partial):**
   - **Total Participation (Double Line `===`):** Every entity instance in the entity set *must* participate in the relationship.
   - **Partial Participation (Single Line `---`):** Some entity instances in the entity set do *not* participate in the relationship.
   - *Classic Example (`Woman <is mother of> Child`):*

```
[ Woman ] -------- < is mother of > ======== [ Child ]
          (Partial)                  (Total)
```

```
 - Not every `Woman` is a mother $\to$ **Partial participation** (single line on
  ↪  `Woman`).
 - Every `Child` biological record must have a mother $\to$ **Total
  ↪  participation** (double line on `Child`).
```

2. **Cardinality Ratios (`1:1`, `1:M`, `M:N`):**
   - **One-to-One (`1:1`):** `Student (1) --- <has> --- (1) Room` (one student occupies at most one room, and one room houses at most one student).
   - **One-to-Many / Many-to-One (`1:M` / `M:1`):** `Student (M) === <has personal tutor> --- (1) Tutor` (each student has exactly one personal tutor, while a tutor advises many students).
   - **Many-to-Many (`M:N`):** `Student (M) === <is taught by> === (N) Tutor` (a student is taught by many tutors, and a tutor teaches many students).

Transforming (Mapping) an ER Model into a Relational Schema

To convert a conceptual ER diagram into a logical Relational Model, apply the following **Six Transformation Rules**:

Rule 1: Mapping a Strong Entity Type

- Create a base relation (table) for each strong entity type.

- Include all its **simple attributes** (and the simple leaf components of any composite attributes).
- Declare the entity's identifier (simple or composite) as the **Primary Key**.

```
ER Entity:      Customer ( _Name_, _DOB_, Address, Job, Salary )
                                |
                                v
Relational Table:
Customer ( Name, DOB, Address, Job, Salary )
           ^^^^^^^^
         Composite PK
```

Rule 2: Mapping a Weak Entity Type

- Create a base relation for the weak entity type containing all its own attributes.
- **Include the Primary Key of the identifying owner entity** as a **Foreign Key** in the weak entity's table.
- The **Primary Key of the weak entity relation** is the **composite combination** of the owner's primary key *plus* the weak entity's own partial key (discriminator).

```
ER Model:
[ Customer (_Name_, _DOB_) ] === << has >> === [[ Account (_Type_, Balance,
↪  OpenDate) ]]
                                                              |
                                                              v
Relational Tables:
Customer ( Name, DOB, Address, Job, Salary )
           ^^^^^^^^
Account  ( Name, DOB, Type, Balance, Opening_date )
           ^^^^^^^^^^^^^^
           Composite PK (where (Name, DOB) is FK -> Customer)
```

Rule 3: Mapping a Binary 1:1 Relationship with Total Participation on One Side

- Suppose `Student ( _Name_ , _DOB_ , Address, Sex )` has **Total Participation (===)** in a `1:1` relationship `<does>` with `Project ( _Title_ , Deadline )` (which has partial participation `---`).
- **Post the Primary Key of the partially participating entity (`Project.Title`) as a Foreign Key into the totally participating entity (`Student`)** (along with any relationship attributes).
- Because every `Student` does a project, the foreign key column `PTitle` in `Student` will never be `NULL`!

```
Student ( Name, DOB, Address, Sex, PTitle )
          ^^^^^^^^                 ^^^^^^
             PK                FK -> Project(Title)
```

```
Project ( Title, Deadline )
          ^^^^^
           PK
```

Rule 4: Mapping a Binary 1:1 Relationship with Partial Participation on Both Sides

- Suppose `Student ( _Name_, _DOB_, Sex )` and `Room ( _Number_, _Hall_ )` both have **Partial Participation (---)** in a `1:1 <has>` relationship.
- You may post the Primary Key of either entity as a Foreign Key into the other table (ideally choosing the side where fewer rows will have `NULL` foreign keys):
  - **Option A (FK placed in `Room`):** `Room ( Number, Hall, Student_Name, Student_DOB )`
  - **Option B (FK placed in `Student`):** `Student ( Name, DOB, Sex, Number, Hall )`

Rule 5: Mapping a Binary 1:M Relationship

- Always **post the Primary Key of the 1-side entity as a Foreign Key into the M-side entity** (along with any attributes attached to the relationship).
- *Why never the reverse?* Placing the M-side key into the 1-side table would require storing a multi-valued list of keys in a single row, violating First Normal Form (1NF)!
- *Example:* `Student (M) = = = <has personal tutor> --- (1) Tutor ( _ Title_ , _ Tutor_Name_, Room )`:

```
Student ( Name, DOB, Address, Sex, T_Title, Tutor_Name )
          ^^^^^^^^                 ^^^^^^^^^^^^^^^^^^
             PK                    FK -> Tutor(Title, Tutor_Name)

Tutor   ( Title, Tutor_Name, Room )
          ^^^^^^^^^^^^^^^^
                PK
```

Rule 6: Eliminating a Binary M:N Relationship (Associative / Junction Table)

- A many-to-many (`M:N`) relationship **cannot** be represented by posting a foreign key into either of the two entity tables without violating 1NF.
- **Step 1 (Conceptual Decomposition):** Decompose the `M:N` relationship (`Subject (M) = == <teaches> --- (N) Tutor`) into **two `1:M` relationships** pointing inwards toward a new intermediate **Associative / Junction Entity** (e.g., `Assignment`):

```
Before (M:N):
[ Subject ] (M) ======= < teaches > ------- (N) [ Tutor ]

After Decomposition (Two 1:M Relationships):
[ Subject ] (1) -- <has> == (M) [ Assignment ] (M) == <assigned> -- (1) [ Tutor ]
```

- **Step 2 (Relational Mapping):** Create a third **Junction Table (`Assignment`)** whose **Composite Primary Key** consists of the foreign keys referencing the primary keys of both participating entities, plus any attributes of the relationship itself:

```
Subject    ( Sub_Code, Sub_Title, Credit )
             ^^^^^^^^
Tutor      ( Title, Tutor_Name, Room )
             ^^^^^^^^^^^^^^^^
Assignment ( Sub_Code, Title, Tutor_Name )
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
             Composite PK (Sub_Code FK -> Subject; (Title, Tutor_Name) FK ->
              ↪  Tutor)
```

Comprehensive SQL Lab: All 20 HR Schema Queries and Solutions

Below are all 20 official `HR` Schema SQL questions from `SQL Questions HR Schema.pdf` alongside the official solutions from `sqlsolutionshr.pdf` and technical explanations.

Filtering, Aggregation, Joins, and Subqueries (Questions 1–10)

1. **Display details of jobs where the minimum salary is greater than 10000:**

```sql
SELECT *
FROM JOBS
WHERE MIN_SALARY > 10000;
```

2. **Display `first_name` and join date (`hire_date`) of the employees who are either IT Programmer (`'IT_PROG'`) or Sales Man (`'SA_MAN'`):**

```sql
SELECT FIRST_NAME, HIRE_DATE
FROM EMPLOYEES
WHERE JOB_ID IN ('IT_PROG', 'SA_MAN');
```

3. **Display details of employee with ID `150` or `160`:**

```sql
SELECT *
FROM EMPLOYEES
WHERE EMPLOYEE_ID IN (150, 160);
```

4. **Display `first_name`, `salary`, `commission_pct`, and `hire_date` for employees with salary less than 10000:**

```sql
SELECT FIRST_NAME, SALARY, COMMISSION_PCT, HIRE_DATE
FROM EMPLOYEES
WHERE SALARY < 10000;
```

5. **Display the `country_id` and number of cities that are in the country:**

```sql
SELECT COUNTRY_ID, COUNT(*)
FROM LOCATIONS
GROUP BY COUNTRY_ID;
```

6. **Display average salary of employees in each department who have a commission percentage (`COMMISSION_PCT IS NOT NULL`):**

```
SELECT DEPARTMENT_ID, AVG(SALARY)
FROM EMPLOYEES
WHERE COMMISSION_PCT IS NOT NULL
GROUP BY DEPARTMENT_ID;
```

7. **Display each region and its countries:**

```
SELECT REGION_NAME, COUNTRY_NAME
FROM REGIONS, COUNTRIES
WHERE COUNTRIES.REGION_ID = REGIONS.REGION_ID
GROUP BY REGION_NAME, COUNTRY_NAME;
```

8. **Display employee `first_name`, `last_name`, and `job_title`:**

```
SELECT FIRST_NAME, LAST_NAME, JOB_TITLE
FROM EMPLOYEES, JOBS
WHERE EMPLOYEES.JOB_ID = JOBS.JOB_ID;
```

9. **Display the `first_name` and `last_name` of employees who earn more than the average salary:**

```
SELECT FIRST_NAME, LAST_NAME
FROM EMPLOYEES
WHERE SALARY > (
    SELECT AVG(SALARY)
    FROM EMPLOYEES
);
```

10. **Display the `first_name` and `last_name` of the employee who has the highest remuneration:**

```
-- Lecturer's solution sheet query:
SELECT FIRST_NAME, LAST_NAME, MAX(SALARY + COMMISSION_PCT)
FROM EMPLOYEES;

-- Strict SQL-mode equivalent using a subquery and IFNULL:
SELECT FIRST_NAME,
       LAST_NAME,
       SALARY + IFNULL(SALARY * COMMISSION_PCT, 0) AS Total_Remuneration
FROM EMPLOYEES
WHERE SALARY + IFNULL(SALARY * COMMISSION_PCT, 0) = (
    SELECT MAX(SALARY + IFNULL(SALARY * COMMISSION_PCT, 0))
    FROM EMPLOYEES
);
```

Subqueries, Pattern Matching, Joins, and DML (Questions 11–20)

11. **Display the details of Departments managed by `'Nancy'` (`first_name`):**

```sql
SELECT *
FROM DEPARTMENTS
WHERE MANAGER_ID IN (
    SELECT EMPLOYEE_ID
    FROM EMPLOYEES
    WHERE FIRST_NAME = 'NANCY'
);
```

12. **Display employees who did not do any job in the past (no record in `JOB_HISTORY`):**

```sql
SELECT *
FROM EMPLOYEES
WHERE EMPLOYEE_ID NOT IN (
    SELECT EMPLOYEE_ID
    FROM JOB_HISTORY
);
```

13. **Display all unique job titles:**

```sql
SELECT DISTINCT JOB_TITLE
FROM JOBS;
```

14. **Display employees where the `first_name` or `last_name` starts with `'S'`:**

```sql
SELECT FIRST_NAME, LAST_NAME
FROM EMPLOYEES
WHERE FIRST_NAME LIKE 'S%'
   OR LAST_NAME  LIKE 'S%';
```

15. **Display details of jobs in the descending order of the title:**

```sql
SELECT *
FROM JOBS
ORDER BY JOB_TITLE DESC;
```

16. **Insert a row into `DEPARTMENTS` table with manager ID `120` and location ID in any location ID for city Tokyo (`1200`):**

```sql
INSERT INTO DEPARTMENTS VALUES (150, 'SPORTS', 120, 1200);
```

17. **Display `department_name` and number of employees in the department:**

```sql
SELECT DEPARTMENT_NAME, COUNT(*)
FROM EMPLOYEES
NATURAL JOIN DEPARTMENTS
GROUP BY DEPARTMENT_NAME;
```

- *Note on `NATURAL JOIN`:* Because `EMPLOYEES` and `DEPARTMENTS` share **both** `DEPARTMENT_ID` and `MANAGER_ID`, `NATURAL JOIN` matches on both columns (returning only employees whose manager is also the head of their department). Joining explicitly on `USING (DEPARTMENT_ID)` counts all employees in each department:

```sql
SELECT d.DEPARTMENT_NAME, COUNT(*)
FROM EMPLOYEES e
JOIN DEPARTMENTS d USING (DEPARTMENT_ID)
```

```
GROUP BY d.DEPARTMENT_NAME;
```

18. **Display `department_name`, manager name (`first_name`), and `city`:**

```
SELECT DEPARTMENT_NAME, FIRST_NAME, CITY
FROM DEPARTMENTS D
JOIN EMPLOYEES E ON (D.MANAGER_ID = E.EMPLOYEE_ID)
JOIN LOCATIONS L USING (LOCATION_ID);
```

19. **Display the details of employees drawing the highest salary in the department:**

```
-- Lecturer's solution sheet query (for a specific department, e.g., 150):
SELECT DEPARTMENT_ID, FIRST_NAME, SALARY
FROM EMPLOYEES
WHERE SALARY = (
    SELECT MAX(SALARY)
    FROM EMPLOYEES
    WHERE DEPARTMENT_ID = 150
);

-- Correlated / IN subquery across all departments:
SELECT DEPARTMENT_ID, FIRST_NAME, LAST_NAME, SALARY
FROM EMPLOYEES
WHERE (DEPARTMENT_ID, SALARY) IN (
    SELECT DEPARTMENT_ID, MAX(SALARY)
    FROM EMPLOYEES
    GROUP BY DEPARTMENT_ID
);
```

20. **Display `job_id` for jobs with average salary more than 10000:**

```
SELECT JOB_ID
FROM EMPLOYEES
GROUP BY JOB_ID
HAVING AVG(SALARY) > 10000;
```

Week 4: DBMS Transaction Management and Concurrency Fundamentals

Motivation for Transaction Management and Concurrency Control

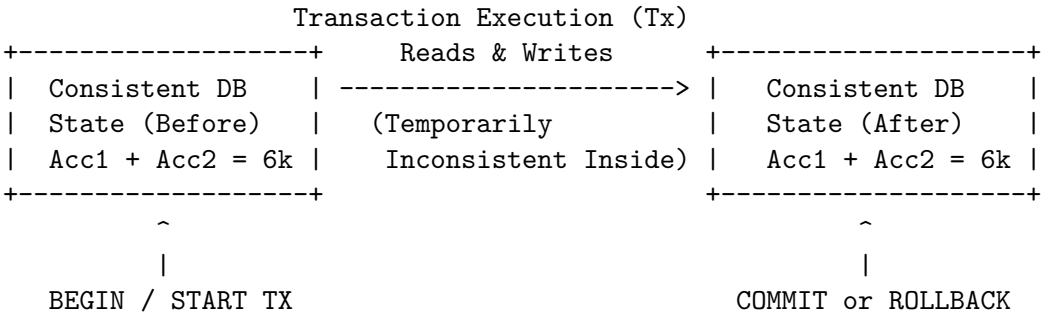
In modern enterprise and cloud environments (such as Netflix, Amazon, or Microsoft Azure), thousands or millions of users read and write to shared databases simultaneously.

- **Single-User vs Multi-User DBMS:** If only one user accessed a database at a time, executing queries strictly one after another would be trivial. However, waiting for each global user to finish before allowing the next user to run would make response times unusable.
- **The Concurrency Challenge:** When multiple users execute concurrent **Read** and **Write** operations against the same records at the same time, uncontrolled interleaving causes severe data corruption (such as **Lost Updates**, **Dirty Reads**, and **Inconsistent Summaries**).

Defining a Database Transaction (Tx)

Logical Unit of Work and State Transitions

A **Transaction (Tx)** is a logical unit of work in a database system consisting of a sequence of one or more **Read** and/or **Write** operations bounded by an explicit start and end marker:



- **Transaction Boundaries in SQL:**
  - **Start:** `BEGIN;` or `START TRANSACTION;`
  - **Body:** A series of SQL `SELECT`, `INSERT`, `UPDATE`, and `DELETE` operations.
  - **Successful Termination (`COMMIT`):** Permanently persists all changes made by the transaction to disk.
  - **Aborted Termination (`ROLLBACK`):** Undoes all changes made by the transaction since `BEGIN`, restoring the database to the exact state it held before the transaction started.
- **Temporary Inconsistency During Execution:**
  - Suppose Transaction T_1 transfers `500` from `Acc1` (initial balance `5000`) to `Acc2` (initial balance `1000`).
  - Before T_1 starts, the total money across both accounts is $5000 + 1000 = 6000$.
  - Immediately after T_1 subtracts `500` from `Acc1` (`Acc1 = 4500`) *before* it adds `500` to `Acc2` (`Acc2 = 1000`), the database is **temporarily inconsistent** ($4500 + 1000 = 5500$).
  - Once T_1 adds `500` to `Acc2` (`Acc2 = 1500`) and executes `COMMIT`, consistency is restored ($4500 + 1500 = 6000$). No external transaction should ever be allowed to observe the intermediate inconsistent state!

Transaction Schedules: Serial, Serializable, and Non-Serializable

Suppose two transactions execute against `Acc1 = 5000` and `Acc2 = 1000`:

- **Transaction T_1:** Transfers `500` from `Acc1` to `Acc2` (`Acc1 = Acc1 - 500`, `Acc2 = Acc2 + 500`).
- **Transaction T_2:** Deposits `250` into `Acc1` (`Acc1 = Acc1 + 250`).

Serial Schedule (No Interleaving)

In a **Serial Schedule**, transactions are executed strictly one after the other with zero overlap (T_1 runs from start to `COMMIT`, and only then does T_2 begin):

Step	Transaction T_1 (Transfer 500)	Transaction T_2 (Deposit 250)	Acc1 in Memory	Acc2 in Memory
1	BEGIN		5000	1000
2	Read(Acc1) $\rightarrow$ 5000		5000	1000
3	Acc1 = Acc1 - 500		4500	1000
4	Write(Acc1) $\rightarrow$ 4500		4500	1000
5	Read(Acc2) $\rightarrow$ 1000		4500	1000
6	Acc2 = Acc2 + 500		4500	1500
7	Write(Acc2) $\rightarrow$ 1500		4500	1500
8	COMMIT		**4500**	**1500**
9		BEGIN	4500	1500
10		Read(Acc1) $\rightarrow$ 4500	4500	1500
11		Acc1 = Acc1 + 250	4750	1500
12		Write(Acc1) $\rightarrow$ 4750	4750	1500
13		COMMIT	**4750**	**1500**

- **Key Property of Serial Execution:** *"Nothing can go wrong if the system executes transactions serially."* Both $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ leave Acc1 = 4750 and Acc2 = 1500.
- **Drawback:** Serial execution eliminates concurrency completely, resulting in the **poorest average throughput and response time** because fast transactions must queue behind slow I/O-bound transactions.

Serializable Schedule (Safe Interleaving)

A **Serializable Schedule** is a schedule in which concurrent transactions **interleave** their operations in time for higher throughput, yet the **net effect on the database is guaranteed to be identical (serially equivalent)** to executing those transactions in some serial order $(T_1, T_2, ...)$:

Step	Transaction T_1	Transaction T_2	Acc1	Acc2
1	BEGIN		5000	1000
2	Read(Acc1) $\rightarrow$ 5000		5000	1000
3	Acc1 = Acc1 - 500		4500	1000
4	Write(Acc1) $\rightarrow$ 4500		4500	1000

Step	Transaction T_1	Transaction T_2	Acc1	Acc2
5		BEGIN	4500	1000
6		Read(Acc1) $\rightarrow$ 4500	4500	1000
7		Acc1 = Acc1 + 250	4750	1000
8		Write(Acc1) $\rightarrow$ 4750	4750	1000
9		COMMIT	4750	1000
10	Read(Acc2) $\rightarrow$ 1000		4750	1000
11	Acc2 = Acc2 + 500		4750	1500
12	Write(Acc2) $\rightarrow$1500| |4750|1500| | 13 |COMMIT| | **4750** | **1500‘**			

- Notice that T_2 interleaved after T_1 had already finished updating Acc1, producing the exact same final balances (Acc1 = 4750, Acc2 = 1500) as the serial schedule!

Non-Serializable Interleaved Schedules (The Lost Update Anomaly)

When two concurrent transactions read the same initial value of Acc1 (5000) before either writes its updated value back, a **Race Condition (Lost Update)** destroys data integrity:

Case A: T_1 Overwrites T_2's Committed Deposit

Step	Transaction T_1 (Transfer 500)	Transaction T_2 (Deposit 250)	Acc1	Acc2
1	BEGIN		5000	1000
2	Read(Acc1) $\rightarrow$ 5000		5000	1000
3		BEGIN	5000	1000
4		Read(Acc1) $\rightarrow$ 5000	5000	1000
5		Acc1 = 5000 + 250	5250	1000
6		Write(Acc1) $\rightarrow$ 5250	5250	1000
7		COMMIT	5250	1000
8	Acc1 = 5000 - 500 *(stale read!)*		4500	1000

Step	Transaction T_1 (Transfer 500)	Transaction T_2 (Deposit 250)	Acc1	Acc2
9	Write(Acc1) → 4500		**4500**	1000
10	Read(Acc2) → 1000; Write(Acc2) → 1500; COMMIT		**4500**	**1500**

- **Outcome:** T_2's 250 deposit has completely vanished! Final Acc1 is 4500 instead of 4750.

Case B: T_2 Overwrites T_1's Committed Debit

- Conversely, if T_2 reads Acc1 = 5000 first, pauses while T_1 runs completely and commits (Acc1 = 4500, Acc2 = 1500), and then T_2 writes Acc1 = 5000 + 250 = 5250 and commits, **T_1's 500 deduction from Acc1 is lost** (Acc1 = 5250, Acc2 = 1500 — creating 500 out of thin air!).

The ACID Properties of Transactions and Their MySQL Implementations

Every reliable transactional DBMS enforces four fundamental guarantees known by the acronym **ACID**:

ACID Property	Core Meaning	How It Is Implemented in MySQL (InnoDB)
Atomicity	**"All or Nothing":** Either every operation inside the transaction completes successfully, or none of them take effect.	Implemented via the **autocommit** system variable, explicit BEGIN / START TRANSACTION, COMMIT, ROLLBACK, and the **Undo Log**.
Consistency	**Correctness of Data:** A transaction must transform the database from one valid, rule-compliant state to another valid state.	Implemented via declarative **Primary Key / Foreign Key constraints**, CHECK constraints, domain data types, **Triggers**, and ON UPDATE / DELETE CASCADE rules.
Isolation	**Independence:** Concurrent transactions execute as if isolated from one another; uncommitted intermediate changes are hidden.	Implemented via the **tx_isolation (transaction_isolation)** variable and concurrency control mechanisms (**Locking**, **Versioning**, and **MVCC**). Involves a direct **trade-off between strictness and performance**.

ACID Property	Core Meaning	How It Is Implemented in MySQL (`InnoDB`)
Durability	**Persistence:** Once a transaction executes `COMMIT`, its changes are permanent and survive power loss or OS crashes.	Implemented via main-memory **DB Buffers** (`InnoDB` Buffer Pool), write-ahead **Redo / Undo Log files on disk**, database **Backups**, **UPS** hardware, and **Replication**.

How `autocommit` Controls Transaction Boundaries in MySQL

MySQL uses a session/global variable named `autocommit` to govern when writes are flushed from memory buffers to permanent storage:

- **`autocommit = 1` (MySQL Default Mode):**
  - Every individual SQL statement (`INSERT`, `UPDATE`, `DELETE`, `DROP`) is treated as a standalone transaction and is **immediately committed to disk** as soon as the statement finishes executing.
  - Because it is already committed, issuing `ROLLBACK;` afterwards has no effect.
- **`autocommit = 0` (Explicit or Implicit via `BEGIN` / `START TRANSACTION`):**
  - When a user executes `BEGIN;` (or `START TRANSACTION;`), MySQL **implicitly suspends autocommit** (`autocommit = 0`) for the duration of that transaction.
  - All subsequent `INSERT`, `UPDATE`, and `DELETE` changes are held in **main memory buffers (`InnoDB` buffer pool / undo logs)** rather than permanently committed.
  - Only when the user issues `COMMIT;` are the buffered changes finalized on disk (or discarded if the user issues `ROLLBACK;`), at which point MySQL automatically restores `autocommit = 1`.

```
-- Inspect current autocommit state (1 = ON by default)
SELECT @@autocommit;

-- Explicitly start a transaction (implicitly sets autocommit = 0 until
 ↪  COMMIT/ROLLBACK)
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE acc_id = 2;
COMMIT; -- Persists changes and restores autocommit = 1
```

Three Concurrency Control Techniques: Overview

To enforce **Isolation** and guarantee serializable schedules without running transactions serially, DBMSs use three main concurrency control strategies:

1. **Locking (Pessimistic Concurrency Control):** Assumes conflicts between concurrent transactions *will* happen frequently, and therefore requires transactions to acquire read or write locks on data items *before* accessing them.

2. **Versioning (Optimistic Concurrency Control):** Assumes conflicts are rare, allowing transactions to execute without acquiring locks and checking for timestamp/version conflicts only at `COMMIT` time.
3. **Multi-Version Concurrency Control (MVCC):** Maintains multiple historical **snapshots (versions)** of modified rows via Undo Logs so read queries never block write transactions and write transactions never block plain readers.

Introduction to Pessimistic Locking and MySQL Storage Engines

Lock Granularity: Database, Table (`MyISAM`), and Row (`InnoDB`)

Locks can be applied at three different levels of **granularity**:

Lock Granularity	Storage Engine Example	Concurrency vs Overhead Trade-Off
Database-Level Lock	Administrative Backup Tools	**Coarsest granularity:** Locks the entire database across all tables. Zero concurrency for writes; used mainly during full system backups.
Table-Level Lock	**MySQL** `MyISAM`	**Coarse granularity:** Locks an entire table at once. Low memory overhead, but poor concurrency because modifying a single row blocks all other users from accessing any other row in that table.
Row-Level + Table-Level Lock	**MySQL** `InnoDB`	**Finest granularity (Multi-granularity):** Locks only the specific affected row(s) in the index, allowing high concurrent throughput across different rows in the same table at the cost of higher memory and CPU lock-tracking overhead.

Explicit Table Locking in `MyISAM` vs Implicit Row Locking in `InnoDB`

- `MyISAM` **(Explicit Table Locks Required):**
  - The programmer must **explicitly** request and release table-level locks using `LOCK TABLES` and `UNLOCK TABLES`.
  - **Strict Access Rule in** `MyISAM`**:** Once a session executes `LOCK TABLES`, that session can **only** read or write the specific tables it locked until it issues `UNLOCK TABLES;`!

```sql
-- Explicit table-level locking in MyISAM
LOCK TABLES emp WRITE, dept READ;

UPDATE emp SET sal = sal + 100 WHERE empno = 7369;
SELECT * FROM dept WHERE deptno = 20;

UNLOCK TABLES;
```

- **InnoDB (Implicit Row-Level + Table-Level Locking):**
  - InnoDB acquires row-level write locks **automatically (implicitly)** whenever a transaction runs INSERT, UPDATE, or DELETE, and releases all held row locks automatically when the transaction executes COMMIT or ROLLBACK.

Lock Modes in InnoDB: Plain SELECT, Shared (S), and Exclusive (X) Locks

Operation / SQL Syntax	Lock Mode Acquired	Effect on Other Concurrent Transactions (TX2)
Plain SELECT ...	**No Locks Taken**	Reads a consistent MVCC snapshot; neither blocks nor is blocked by any S or X lock!
SELECT ... LOCK IN SHARE MODE; *(or FOR SHARE in MySQL 8+)*	**Read Lock (Shared / S-Lock)**+ Table-level IS Lock	Other transactions **can** read the row and **can** also acquire their own S-Lock (SELECT ... LOCK IN SHARE MODE), **but NO transaction can acquire a Write (X) Lock** (UPDATE, DELETE, FOR UPDATE) on that row until all S-locks are released.
UPDATE ... / DELETE ... / INSERT ...or SELECT ... FOR UPDATE;	**Write Lock (Exclusive / X-Lock)**+ Table-level IX Lock	**Strict exclusivity:** Prevents any other transaction from modifying the row OR placing **any lock (S or X)** on that row until the holding transaction commits or rolls back. (Plain unlocked SELECT still works via MVCC snapshot).

Lock Compatibility Matrix (Row Level)

Requested Mode \ Held Mode	**Unlocked (None)**	**Shared (S / Read Lock)**	**Exclusive (X / Write Lock)**
Plain SELECT (Snapshot)	Allowed	Allowed	Allowed (reads Undo Log snapshot)

Requested Mode \ Held Mode	Unlocked (None)	Shared (S / Read Lock)	Exclusive (X / Write Lock)
Shared (S / LOCK IN SHARE MODE)	Granted	**Granted (Shared)**	**BLOCKED (Waits)**
Exclusive (X / FOR UPDATE, UPDATE, DELETE)	Granted	**BLOCKED (Waits)**	**BLOCKED (Waits)**

How InnoDB Locks Rows via Index Files in Main Memory

A foundational architectural principle of InnoDB is:

InnoDB does NOT lock physical table rows on disk directly — InnoDB always locks entries inside the Index File in Main Memory (Record Locks).

Because InnoDB locks index records, the presence or absence of an index on the WHERE clause column dramatically alters how many rows get locked during an equality (= or !=) write operation:

```
CASE 1: Write on PRIMARY KEY (WHERE Empno = 5000)
+-----------------------------------+
| Primary Index in Main Memory      |
|   [ 4900 ]                        |
|   [ 5000 ]  <-- ONLY 1 Record     |
|   [ 5200 ]      Locked (X-Lock)!  |
+-----------------------------------+

CASE 2: Write on INDEXED Secondary Column (WHERE Ename = 'Smith')
+-----------------------------------+      +-----------------------------------+
| Secondary Index (Ename) in Memory |      | Primary Index (Empno) in Memory   |
|   [ 'Allen' ]                     |      |   [ 4900 ]                        |
|   [ 'Smith' ] <-- Locked (X-Lock) | ---> |   [ 5000 ] <-- Corresponding PK   |
|   [ 'Ward'  ]                     |      |   [ 5200 ]     Record Also Locked |
+-----------------------------------+      +-----------------------------------+

CASE 3: Write on NON-INDEXED Column (WHERE age = 22)
+-------------------------------------------------------------------------------+
| NO Index on 'age'! InnoDB must scan the entire Primary Index and              |
| LOCK EVERY SINGLE ROW AND GAP IN THE ENTIRE TABLE!                            |
+-------------------------------------------------------------------------------+
```

1. **Equality Write on Primary Key (WHERE Empno = 5000):**
   - MySQL locates 5000 directly in the **Primary Key Index** in main memory and places a **Record Lock (locks rec but not gap) on that single row only**.
2. **Equality Write on an Indexed Non-Key Column (WHERE Ename = 'Smith'):**
   - MySQL locates 'Smith' in the **Secondary Index** in main memory, locks that secondary index entry, and locks the corresponding record in the **Primary Key Index**. All other

rows remain completely free for concurrent updates.

3. **Equality Write on a Non-Indexed Column (`WHERE age = 22`):**
   - Because there is no index on `age`, `InnoDB` cannot determine which index entries match without doing a full scan, so it **locks every single row and gap across the entire table**!
   - **Key Takeaway:** Forgetting to index columns used in `UPDATE` / `DELETE` / `SELECT ... FOR UPDATE WHERE` clauses silently degrades `InnoDB`'s fine-grained row locking into a full-table lock!

Week 5: Advanced InnoDB Locking, Versioning, MVCC, and Practical Concurrency Lab

Range Queries, Phantom Rows, Gap Locks, and Next-Key Locks

Why Record Locks Alone Fail for Range Queries (`>`, `<`, `>=`, `<=`, `BETWEEN`)

Suppose a table `EMP` currently contains four employee numbers in its Primary Key index: `5000`, `5200`, `5300`, and `5400`.

- Suppose Transaction `TX1` executes a range lock:

```
SELECT * FROM emp WHERE empno > 5200 FOR UPDATE;
```

- If `InnoDB` only locked the existing index records `5300` and `5400`, another concurrent transaction `TX2` could execute `INSERT INTO emp VALUES (5250, ...)` into the empty space between `5200` and `5300`!
- If `TX1` then re-ran its `SELECT` (or `UPDATE`), a brand-new row (`5250`) would suddenly appear inside its locked range—an isolation anomaly known as a **Phantom Row (Phantom Read)**.

Gap Locks, `infimum`, and `supremum`

To prevent Phantom Rows, `InnoDB` locks not only the index records themselves, but also the **Gaps** between index entries where new keys could potentially be inserted.

Every `InnoDB` index page contains two special virtual boundary records created automatically by the storage engine:

- **`infimum` record:** A virtual marker representing the lower bound **before the very first record** in the index (beginning of the table).
- **`supremum` record:** A virtual marker representing the upper bound **after the very last record** in the index (end of the table).

Thus, `InnoDB` recognises **three categories of Gaps** in an index file (`5000`, `5200`, `5300`, `5400`):

```
[ infimum ] <--- Gap 1 (Before First Record: < 5000) ---> [ 5000 ]
  [ 5000 ]  <--- Gap 2a (Between Records: 5001..5199) --> [ 5200 ]
  [ 5200 ]  <--- Gap 2b (Between Records: 5201..5299) --> [ 5300 ]
  [ 5300 ]  <--- Gap 2c (Between Records: 5301..5399) --> [ 5400 ]
  [ 5400 ]  <--- Gap 3 (After Last Record: > 5400) -----> [ supremum ]
```

1. **Gap before the first record:** Between `infimum` and `5000` (prevents inserting `Empno < 5000`).
2. **Gaps between existing index records:** Between `5000` and `5200`, `5200` and `5300`, and `5300` and `5400`.
3. **Gap after the last record:** Between `5400` and `supremum` (prevents inserting `Empno > 5400`).

Important Exception: Gap Locks are NOT required for unique equality lookups (`WHERE empno = 5200` on a unique/primary key). Because primary keys are unique, no second row with `5200` could ever be inserted anyway, so `InnoDB` takes only a pure **Record Lock (`locks rec but not gap`)**.

Next-Key Locks (Record Lock + Preceding Gap Lock)

A **Next-Key Lock** is `InnoDB`'s default row-locking unit for range scans: it combines a **Record Lock** on an index entry *together with* a **Gap Lock on the gap immediately preceding that index entry**.

Let us trace two concrete examples on the `emp` index (`5000, 5200, 5300, 5400`):

Example 1: Upper-Bound Range (`WHERE empno > 5350 FOR UPDATE`)

```
SELECT * FROM emp WHERE empno > 5350 FOR UPDATE;
```

- **What gets locked by `InnoDB`?**
  1. **Next-Key Lock on `5400`:** Locks the **entire gap between `5300` and `5400`** (even values like `5320` that are ≤ 5350, because `InnoDB` locks whole gaps between existing keys, never partial gaps!) *plus* the **index record `5400`**.
  2. **Next-Key Lock on `supremum`:** Locks the **gap after `5400` up to `supremum`** (preventing any insert > 5400).

Example 2: Lower-Bound Range (`WHERE empno < 5350 FOR UPDATE`)

```
SELECT * FROM emp WHERE empno < 5350 FOR UPDATE;
```

- **What gets locked by `InnoDB`?**
  1. The gap before `5000` (from `infimum`) and **Record `5000`**.
  2. The gap `(5000, 5200)` and **Record `5200`**.
  3. The gap `(5200, 5300)` and **Record `5300`**.
  4. **The gap `(5300, 5400)` AND Record `5400` (Next-Key Lock on `5400`)** — because to protect the gap `(5300, 5350)` from phantom inserts (such as `5320`), `InnoDB` places a Next-Key Lock on the next existing index entry (`5400`)!

Specialized InnoDB Locks: Auto-Increment, Insert Intention, and Table Intention Locks

Auto-Increment Locks (AUTO-INC)

- When a table defines an AUTO_INCREMENT primary key, concurrent INSERT statements must never receive the same generated key value.
- InnoDB uses a special **table-level AUTO-INC lock**:
  - When TX1 executes an INSERT, it briefly acquires the AUTO-INC lock to allocate the next sequence number, places a row-level Write (X) lock on the newly inserted record, and **releases the AUTO-INC lock immediately at the end of the INSERT statement** (rather than holding it until COMMIT), allowing other transactions to continue inserting new auto-increment rows without waiting for TX1 to finish!

Insert Intention Locks

- An **Insert Intention Lock** is a specialized type of **Gap Lock** set by INSERT operations prior to row insertion (reported in SHOW ENGINE INNODB STATUS as lock_mode X insert intention waiting when blocked by an existing gap/next-key lock).
- **Why It Exists (Concurrent Inserts into the Same Gap):**
  - Suppose a gap exists between keys 5200 and 5300, and no range lock is currently held on that gap.
  - If TX1 inserts 5250 and TX2 simultaneously inserts 5260, both transactions acquire **Insert Intention Locks** on the (5200, 5300) gap.
  - Because Insert Intention Locks **do not block each other** as long as the inserted keys are at **different positions** (5250 != 5260), both inserts succeed concurrently without waiting!
  - However, if both try to insert the *same* key 5250, the second transaction is blocked on a duplicate-key lock check.

Table-Level Intention Locks (IS and IX)

- InnoDB supports multi-granularity locking (both row-level locks and full table-level locks such as LOCK TABLES ... WRITE or ALTER TABLE / DROP TABLE).
- **The Problem:** If Transaction TX2 requests a full table-level lock on a 10-million-row table, how can InnoDB quickly know whether some Transaction TX1 currently holds a row lock on row #8,432,109 without scanning all 10 million index records?
- **The Solution (Table-Level Intention Flags):**
  - Before InnoDB grants a row-level **Shared (S)** lock to a transaction, it first places a lightweight **Intention Shared (IS)** flag at the **table level**.
  - Before InnoDB grants a row-level **Exclusive (X)** lock to a transaction, it first places an **Intention Exclusive (IX)** flag at the **table level**.
  - **Crucial Property:** Table-level IS and IX locks **never block row-level locks** from other transactions (IX is compatible with IX and IS at the table level—conflicts are resolved at the individual row level). They **only block full-table locks** (DROP TABLE, ALTER TABLE, LOCK TABLES WRITE)!

Locking Pathologies: Deadlocks, Lost Updates, and Starvation

Deadlocks (Circular Wait)

A **Deadlock** occurs when two or more transactions each hold a lock that the other transaction needs, creating a permanent **circular wait** where neither transaction can proceed:

```
Transaction 1 (TX1)                    Transaction 2 (TX2)
-------------------                    -------------------
1. Locks Row 10 (X-Lock)
2. Locks Row 20 (X-Lock)

                                       3. Locks Row 30 (X-Lock)
                                       4. Requests Lock on Row 20
                                          --> WAITS for TX1!
5. Requests Lock on Row 30
   --> WAITS for TX2! (DEADLOCK CYCLE!)
```

Two Approaches to Handling Deadlocks

Strategy	Mechanism	Pros and Cons
Deadlock Prevention	1. **Pre-declaration:** Require every transaction to acquire **all** locks it will need at the very beginning before executing.2. **Back-Off and Re-Try:** If any requested lock cannot be acquired immediately on the first attempt, release all locks currently held, wait briefly, and retry from scratch.	**Pros:** Deadlocks can never occur.**Cons:** Often impossible to predict upfront which rows a complex transaction will need; pre-locking ties up many rows for longer than necessary and hurts concurrency.
Deadlock Detection & Resolution	Allow transactions to acquire locks dynamically as needed. The DBMS periodically runs a **Wait-For Graph cycle detection algorithm**. When a cycle is detected, the DBMS selects a **victim** and forces an automatic `ROLLBACK`.	**Pros:** Maximises concurrency during normal operation (`InnoDB` default).**Cons:** Runtime CPU overhead for cycle checking and rollback cost for the victim transaction.

- **How `InnoDB` Chooses the Deadlock Victim ("Lowest Weight"):** When `InnoDB` detects a deadlock cycle, it automatically rolls back the transaction with the **lowest weight**—defined as the transaction that has modified/inserted/deleted the **fewest rows** (fewest undo log entries), thereby minimising the work required to roll it back!

Lost Updates Under Late Locking

- Even in a locking system, if two transactions read a record **without acquiring a lock** (`SELECT balance FROM accounts WHERE id = 1`), compute a new value in application memory (`balance = balance - 100`), and only acquire an `X`-lock at the final `UPDATE` step, the second update will overwrite the first.
- **Fix:** Either perform atomic relative updates (`UPDATE accounts SET balance = balance - 100 WHERE id = 1`) or acquire an exclusive read lock upfront using `SELECT ... FOR UPDATE`.

Transaction Starvation

- **Definition: Starvation (Livelock)** occurs when a waiting transaction is perpetually bypassed and never acquires all the locks it needs because the lock manager's scheduling policy repeatedly grants locks to other competing transactions.
- **How DBMSs Prevent Starvation:**
  1. **FIFO (First-In, First-Out) Lock Queues:** Competing lock requests on the same resource are queued in arrival order.
  2. **Dynamic Priority Aging:** Gradually increasing the scheduling priority of any transaction the longer it has been waiting for a lock.

Optimistic Concurrency Control: Versioning and MVCC

Versioning (Optimistic Concurrency Control)

Unlike Pessimistic Locking (which assumes conflicts will happen and locks rows upfront), **Versioning** assumes conflicts between transactions are rare:

- **How Versioning Works:**
  1. **Read & Execution Phase (No Locks):** Transactions read and modify data freely in private workspace buffers without acquiring any locks. **Deadlocks are impossible** because no transaction ever waits on a lock.
  2. **Timestamp / Start-Time Visibility:** Each transaction is assigned a start timestamp and only sees database state committed prior to its start time.
  3. **Validation & Commit Phase:** When a transaction issues `COMMIT`, the DBMS checks whether another transaction modified any of the same data items after this transaction started:
     - If **no conflict** occurred, the transaction commits immediately.
     - If a **conflict** is detected, the **oldest transaction gets priority** and the conflicting younger transaction is automatically **rolled back and restarted**.
- **Trade-Off:** Extremely fast when write contention is low (read-heavy workloads), but suffers high rollback/restart overhead under heavy write contention on hot rows.

Multi-Version Concurrency Control (MVCC) in `InnoDB`

`InnoDB` combines row-level locking for writers with **Multi-Version Concurrency Control (MVCC)** for readers so that **plain `SELECT` queries never block `UPDATE`/`DELETE` writers, and**

writers never block plain `SELECT` readers.

The Three Hidden System Columns on Every `InnoDB` Row Behind the scenes, `InnoDB` silently prepends **three hidden metadata columns** to every single row stored in a table:

Hidden Column Name	Size	Purpose in `InnoDB` MVCC
`TR_Id (DB_TRX_ID)`	6 Bytes	Stores the monotonically increasing **Transaction ID** of the last transaction that inserted or updated this row.
`Rollback_PTR (DB_ROLL_PTR)`	7 Bytes	Stores a **pointer to the previous version** of this row inside the **Undo Log** segment (`ibdata1` / undo tablespace).
`ROW_ID (DB_ROW_ID)`	6 Bytes	Hidden auto-incrementing row ID used as the clustered index key if the table was created without a user-defined Primary Key.

```
Physical InnoDB Row Layout on Disk / Buffer Pool:
+--------+-------------+--------+---------+---------+--------+
| TR_Id  | Rollback_PTR | ROW_ID |  EmpNo  |  EName  |  Sal   |
| (6 B)  |    (7 B)     | (6 B)  | (User)  | (User)  | (User) |
+--------+-------------+--------+---------+---------+--------+
  Hidden System Columns (19 B)     User-Defined Schema Columns
```

How `InnoDB` Reconstructs Consistent Snapshots ("Worming" Through the Undo Log) Suppose a row (`EmpNo = 7369`) in the main database tablespace (`ibdata1`) has been updated successively by transactions `Tx 86`, `Tx 89`, `Tx 92`, and `Tx 100`, and is now modified by `Tx 110`, `Tx 125`, and `Tx 135`:

```
Main Tablespace (ibdata1)            Undo Log Chain (Historical Versions)
+-----------------------+            +-----------------------+
| Current Row (Emp 7369)|            | Undo Record (Tx 100)  |
| TR_Id = 135           | ---------> | TR_Id = 100           |
| Rollback_PTR          |            | Rollback_PTR          |
+-----------------------+            +-----------------------+
                                              |
                                              v
                                     +-----------------------+
                                     | Undo Record (Tx 92)   |
                                     | TR_Id = 92            |
                                     | Rollback_PTR          |
                                     +-----------------------+
                                              |
```

```
                    v
+-----------------------+
| Undo Record (Tx 86)   |
| TR_Id = 86            |
| Rollback_PTR = NULL   |
+-----------------------+
```

- **Scenario:** Suppose a long-running reader transaction with **`Tr_Id = 88`** (or `Tr_Id = 100`) issues a plain `SELECT * FROM emp WHERE empno = 7369;`.
- **Step-by-Step Snapshot Reconstruction:**
  1. `InnoDB` inspects the current row in `ibdata1` and sees `TR_Id = 135`.
  2. Because `135 > 88` (the row was modified *after* Transaction `88` began), Transaction `88` is **not allowed** to see the `TR_Id = 135` version!
  3. Instead of blocking or waiting for locks, `InnoDB` follows the row's **`Rollback_PTR`** into the **Undo Log** ("worming" backwards through `Tx 100` → `Tx 92` → `Tx 86`) and applies the undo log records inside **Main Memory (`Db Buffers`)** until it reaches the version with **`TR_Id = 86`** (≤ 88).
  4. `InnoDB` returns that reconstructed historical **Snapshot** to Transaction `88` immediately with **zero lock contention**!
- **Undo Log Retention and Purging:**
  - `InnoDB` must keep historical undo log chains on disk for as long as any active open transaction might still require an older snapshot.
  - Once all transactions older than a given undo record have committed, `InnoDB`'s background purge thread safely discards those obsolete undo log entries and reclaims the space.

Practical Concurrency Lab: `7. Locking.sql` and Live `InnoDB` Lock Monitor Analysis

In the practical concurrency lab (`7. Locking.sql` and accompanying video walkthrough), we configure WAMP's MySQL server and use **three concurrent MySQL console windows** (`TX1`, `TX2`, and a 3rd `Monitor` terminal) to inspect real-time lock structures in `InnoDB`.

Lab Setup and `my.ini` Configuration

1. **Increasing the Lock Wait Timeout (`my.ini`):**
   - By default in MySQL, `innodb_lock_wait_timeout` is only `50` seconds (or `5` seconds in some dev setups). Edit `C:\wamp64\bin\mysql\mysql8.x.x\my.ini` to set:

```
innodb_lock_wait_timeout = 50
```

   - This gives a 50-second window while a conflicting query in `TX2` is hanging so we can switch to Terminal 3 and run `SHOW ENGINE INNODB STATUS;` before `TX2` times out with: `ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction`.

2. **Enabling the Detailed `InnoDB` Lock Monitor:**

```
CREATE DATABASE IF NOT EXISTS locktest;
USE locktest;
```

```
-- Create the lock monitor table and enable lock output in SHOW ENGINE INNODB
 ↪  STATUS
CREATE TABLE innodb_lock_monitor (a INT) ENGINE=INNODB;
SET GLOBAL innodb_status_output = ON;
SET GLOBAL innodb_status_output_locks = ON;
```

Detailed Walkthrough of All 9 Locking Lab Experiments

Experiment 1: Non-Conflicting vs Conflicting Equality Record Locks on a Primary Key

- **Setup:** Table `T (Field1 INT PRIMARY KEY, Field2 INT)` populated with three rows: `(2, 2)`, `(4, 4)`, `(8, 8)`.
- **Step 1A (Non-Conflicting Rows):**
  - `TX1` executes:

```
BEGIN;
SELECT * FROM T WHERE Field1 = 8 FOR UPDATE;
```

- `TX2` executes:

```
BEGIN;
SELECT * FROM T WHERE Field1 = 2 FOR UPDATE;
```

- **Result:** Both succeed immediately! `SHOW ENGINE INNODB STATUS` shows that `TX1` holds 1 table-level `IX` lock and 1 row-level lock (`lock_mode X locks rec but not gap` on `Field1 = 8`), while `TX2` holds `lock_mode X locks rec but not gap` on `Field1 = 2`.
- **Step 1B (Conflicting Request on `Field1 = 2`):**
  - While `TX2` still holds the lock on `Field1 = 2`, `TX1` executes:

```
SELECT * FROM T WHERE Field1 = 2 FOR UPDATE;
```

- **Result: `TX1` blocks (hangs)** for 50 seconds waiting for `TX2` to release `Field1 = 2`, and `SHOW ENGINE INNODB STATUS` reports `WAITING FOR THIS LOCK TO BE GRANTED: RECORD LOCKS ... lock_mode X locks rec but not gap waiting`.

Experiment 2: Locking Beyond the Table Boundaries (`supremum` and `infimum`)

- **Setup:** Same table `T` with rows `(2, 2)`, `(4, 4)`, `(8, 8)`.
- **Step 2A:**
  - `TX1` runs `SELECT * FROM T WHERE Field1 > 8 FOR UPDATE;` → Locks the **virtual `supremum` record** (`1 lock struct, 1 row lock: supremum pseudo-record`), preventing any insert > 8.
  - `TX2` runs `SELECT * FROM T WHERE Field1 < 2 FOR UPDATE;` → Locks the **gap before record 2 (`lock_mode X locks gap before rec` on `Field1 = 2` / `infimum`)**, preventing any insert < 2.
- **Step 2B (Plain `SELECT` vs `INSERT` into Locked Gap):**

  - In `TX1`, running a plain `SELECT * FROM T WHERE Field1 = 2;` **succeeds immediately** (MVCC snapshot read takes zero locks).
  - In `TX1`, running `INSERT INTO T VALUES (1, 1);` **hangs and times out** because key `1` falls inside the `< 2` gap locked by `TX2`!

Experiment 3: Range Next-Key Locking (`WHERE Field1 > 2 FOR UPDATE`)

- **Setup:** Table `T` has `(2, 2)`, `(4, 4)`, `(8, 8)`.
- **`TX1` executes:**

```
BEGIN;
SELECT * FROM T WHERE Field1 > 2 FOR UPDATE;
```

- **Monitor Output (`SHOW ENGINE INNODB STATUS`):** `TX1` holds **3 row locks**:
  1. Next-Key Lock on `Field1 = 4` (record `4` + gap `(2, 4)`).
  2. Next-Key Lock on `Field1 = 8` (record `8` + gap `(4, 8)`).
  3. Next-Key Lock on `supremum` (gap `> 8`).
- When `TX2` attempts `SELECT * FROM T WHERE Field1 = 8 FOR UPDATE;`, it blocks immediately because record `8` is inside `TX1`'s locked range.

Experiment 4: Inserting into Locked Gaps When Primary Key Values Have Holes

- **Setup:** Table `T` is recreated with 8 rows: Primary Keys `2, 3, 5, 6, 7, 8, 9, 10` (notice that **`1` and `4` are missing**!).
- **`TX1` executes:**

```
BEGIN;
SELECT * FROM T WHERE Field1 > 1 FOR UPDATE;
```

- **Monitor Output:** `TX1` holds **9 row locks** (all 8 existing data records `2, 3, 5, 6, 7, 8, 9, 10` together with the gaps in front of each—including the gap `(3, 5)` where `4` is missing and the gap `< 2` where `1` is missing—plus `supremum`).
- **`TX2` attempts:**

```
BEGIN;
-- Hangs! (lock_mode X insert intention waiting on gap before 5)
INSERT INTO T VALUES (4, 16);
-- Hangs! (gap before 2 is locked by > 1 range scan)
INSERT INTO T VALUES (1, 2);
```

Experiment 5: Boundary Difference Between `>= 2` and `> 1`

- **Setup:** Same 8-row table `T` (`2, 3, 5, 6, 7, 8, 9, 10`).
- **`TX1` executes:**

```
BEGIN;
SELECT * FROM T WHERE Field1 >= 2 FOR UPDATE;
```

- **What Changes?** Because `Field1 = 2` is an exact boundary hit on an existing unique Primary Key, `InnoDB` locks record `2` (`locks rec but not gap`) and Next-Key locks `3..10` and `supremum`, **leaving the gap before 2 (< 2) UNLOCKED**!
- Consequently, when `TX2` executes `INSERT INTO T VALUES (1, 1);`, it **succeeds immediately**, whereas `INSERT INTO T VALUES (4, 16);` still blocks (because `(3, 5)` is inside the `>= 2` range)!

Experiment 6: Boundary Test with `WHERE Field1 > 5 FOR UPDATE`

- **`TX1` executes:**

```
BEGIN;
SELECT * FROM T WHERE Field1 > 5 FOR UPDATE;
```

- **Monitor Output:** `TX1` locks `6, 7, 8, 9, 10` (and the gaps before them, starting after `5`) plus `supremum` (6 row locks total).
- **`TX2` executes:**
  - `INSERT INTO T VALUES (4, 16);` → **Succeeds!** (Gap `(3, 5)` is below `5` and therefore unlocked).
  - `INSERT INTO T VALUES (1, 2);` → **Succeeds!** (Gap `< 2` is also unlocked).

Experiment 7: `DELETE`, `INSERT`, and Duplicate-Key Shared Lock Contention

- **Part A (`DELETE` on Primary Key Takes Only a Record Lock):**
  - `TX1` runs `DELETE FROM T WHERE Field1 = 5;` → Acquires **1 row lock** (`lock_mode X locks rec but not gap` on `Field1 = 5`, with `1 undo log entry`).
  - Similarly, if `TX1` runs `DELETE FROM T WHERE Field1 = 2;`, it locks only record `2` (`locks rec but not gap`), so `TX2` can still execute `INSERT INTO T VALUES (1, 1);` into the unlocked gap before `2`!
- **Part B (Concurrent `INSERT` Duplicate-Key Detection via `S`-Lock):**
  - `TX2` runs `INSERT INTO T VALUES (11, 2048);` and `INSERT INTO T VALUES (12, 4096);` without committing (`TX2` holds `X` locks on `11` and `12`).
  - Now `TX1` attempts to insert the exact same primary key:

```
INSERT INTO T VALUES (11, 2048);
```

- **Result:** `TX1` detects the uncommitted duplicate key `11` and requests a **Shared (`S`) lock** on `Field1 = 11`, which **blocks** waiting for `TX2` to either `COMMIT` (causing `TX1` to fail with Duplicate Key Error) or `ROLLBACK` (allowing `TX1`'s insert to proceed)!

Experiment 8: Full-Table Primary Key `UPDATE` vs Non-Key `UPDATE`

- **`TX1` executes:**

```
BEGIN;
UPDATE T SET Field1 = Field1 + 10;
```

- Because TX1 modifies every Primary Key in the table, InnoDB locks all 8 original index records, all 8 newly created index records (12..20), and supremum (**17 row locks total**, 16 undo log entries), completely blocking TX2 from running UPDATE T SET Field2 = Field2 + 20;.

Experiment 9: The Dramatic Effect of Secondary Indexes on Write Concurrency This final experiment proves why indexing foreign keys and frequently filtered WHERE columns is mandatory in production InnoDB databases:

- **Phase A: Table T (Field1 PK, Field2) with 10 rows (1, 2)..(10, 1024) and NO Index on Field2:**
  1. TX1 executes an equality lock on the unindexed column Field2:

```
BEGIN;
SELECT * FROM T WHERE Field2 = 2 FOR UPDATE;
```

  2. **Monitor Output:** Even though only 1 row has Field2 = 2, because Field2 has no index, TX1 locks **all 11 records in the table** (all 10 primary key rows + gaps + supremum)!
  3. When TX2 tries to lock an unrelated row (SELECT * FROM T WHERE Field1 = 8 FOR UPDATE;), TX2 is **completely blocked**!

- **Phase B: Adding an Index on Field2 (CREATE INDEX index_field2 ON T(Field2);):**
  1. We create a secondary index on Field2:

```
CREATE INDEX index_field2 ON T(Field2);
```

  2. Now TX1 runs the exact same query:

```
BEGIN;
SELECT * FROM T WHERE Field2 = 2 FOR UPDATE;
```

  3. **Monitor Output:** TX1 now holds **only 3 row locks** instead of 11:
     - 1 Next-Key Lock on Field2 = 2 in index_field2.
     - 1 Record Lock (locks rec but not gap) on Field1 = 1 in the PRIMARY index.
     - 1 Gap Lock (locks gap before rec) before Field2 = 4 in index_field2 (to prevent another row with Field2 = 2 from being inserted).
  4. Now when TX2 runs SELECT * FROM T WHERE Field1 = 8 FOR UPDATE;, **TX2 succeeds immediately without blocking**!

Week 6: Codd's 12 Rules for Relational Database Management Systems and Data Quality

Historical Context: Dr Edgar Frank "Ted" Codd and the 1985 Rules

Biographical and Scientific Milestones of E. F. Codd (1923–2003)

- **Early Life and Academic Training:** Born on the Isle of Portland in Dorset, England (1923); studied Mathematics and Chemistry at Exeter College, Oxford, and served as a Flight Lieutenant in the Royal Air Force (RAF) Coastal Command during World War II.

- **IBM Career and PhD:** Moved to the United States after WWII, joined **IBM** in 1949 as a mathematical programmer on the Selective Sequence Electronic Calculator (SSEC) and IBM 701, and completed his PhD in Communication Sciences at the University of Michigan (1965) on self-reproducing cellular automata.
- **Invention of the Relational Model (1969–1970):** Working at the **IBM San Jose Research Laboratory**, Codd published his internal IBM Research Report (`RJ599`, 1969) followed by his landmark June 1970 ACM paper, *"A Relational Model of Data for Large Shared Data Banks"*, and subsequently defined **First, Second, and Third Normal Forms (1NF, 2NF, 3NF)**.
- **Turing Award and Later Versions of the Model:** Received the **1981 ACM Turing Award**. In 1990, Codd published *The Relational Model for Database Management: Version 2 (RM/V2)* comprising **333 features/rules**, followed by work on *RM/V3* extending logic to **Four-Valued Logic (4VL)** (distinguishing between *"Missing but Applicable"* `A-mark` and *"Missing and Inapplicable"* `I-mark`).

Why Codd Formulated the 12 Rules (October 1985 *Computerworld* Articles)

By the mid-1980s, "Relational" had become a lucrative marketing buzzword. Numerous commercial database vendors took legacy hierarchical, network, or inverted-list engines, bolted on a superficial tabular query interface, and marketed them as "Relational DBMSs".

1. **"Pseudo-Relational" vs "Truly-Relational" DBMS:**
   - Many commercial products allowed users to view tables, yet still relied on physical pointers underneath or lacked declarative catalog-enforced integrity constraints.
2. **Performance Benchmarking Shortcuts at the Expense of Data Quality:**
   - Enforcing relational integrity rules in the system catalog adds runtime overhead before updates. Some vendors deliberately omitted catalog-level integrity enforcement so their databases would appear faster in transaction benchmarks—sacrificing **Data Quality** for raw speed.
3. **Why Practice Needs a Theoretical Framework:**
   - As highlighted in the lecture transcript via former Taoiseach Garret FitzGerald's famous quip (*"That's all right in practice, but will it work in theory?"*), a database system that "seems to work in practice" without a rigorous mathematical foundation inevitably fails when edge cases, concurrent updates, or schema evolutions occur. Codd published his **Rule 0 + 12 Rules** in a two-part *Computerworld* series (October 1985) as a strict yardstick to test whether a DBMS is genuinely relational and guarantees **Information Quality**.

Codd's Rule 0 and the 12 Rules in Detail

Rule 0: The Foundation Rule

Rule 0: *For any system that is advertised as, or claimed to be, a relational data base management system, that system must be able to manage data bases entirely through its relational capabilities.*

- **Meaning:** A DBMS cannot claim to be relational if administrators or applications must drop down to non-relational record-at-a-time pointer manipulation or bypass the relational engine to perform data definition, data manipulation, or administration.

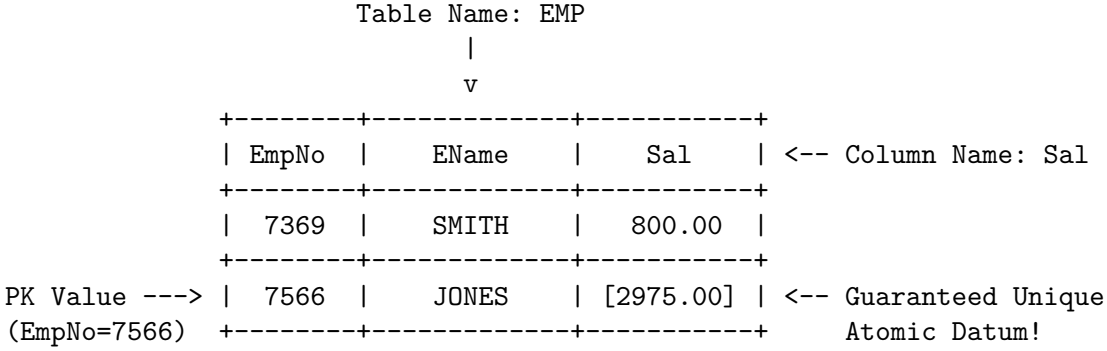
Rule 1: The Information Rule

Rule 1: *All information in a relational data base is represented explicitly at the logical level and in exactly one way — by values in tables.*

- **Theoretical Meaning:**
  - The **Relation (Table)** is the **sole logical data structure** in a relational database.
  - Every piece of information—whether user business data or system metadata—must be stored as atomic attribute values sitting at the intersection of rows (tuples) and columns (attributes).
  - Nothing is encoded in hidden physical pointers, record ordering, or linked lists (unlike hierarchical and network models).
- **Practical Implications in MySQL:**
  - Live structural modifications can be performed at the logical level using SQL DDL (`ALTER TABLE emp ADD COLUMN ...`) without rebuilding physical pointer chains.
  - Even relationships between tables (`EMP.DEPTNO` referencing `DEPT.DEPTNO`) are represented purely by matching **values in tables**, never by hard-coded memory/disk addresses!

Rule 2: The Guaranteed Access Rule

Rule 2: *Each and every datum (atomic value) in a relational data base is guaranteed to be logically accessible by resorting to a combination of table name, primary key value and column name.*



- **Theoretical Meaning:**
  - Because every row in a relation is uniquely identified by its **Primary Key** and every column has a unique **Attribute Name** within that table, the 3-tuple:

$$(\text{Table Name}, \text{Primary Key Value}, \text{Column Name})$$

acts as a universal logical coordinate system capable of pinpointing any
↪ individual atomic value in the entire database.

- Users never need to know array indices, physical disk offsets, or pointer chains, nor do they need to scan sequentially through a file to reach a value.
- **Connection to Primary Keys:** Rule 2 directly restates why **every relational table must possess a Primary Key** and why all stored attribute values must be **atomic (1NF)**.

Rule 3: Systematic Treatment of Null Values

Rule 3: *Null values (distinct from the empty character string or a string of blank characters and distinct from zero or any other number) are supported in fully relational DBMS for representing missing information and inapplicable information in a systematic way, independent of data type.*

- **What `NULL` Is and Is NOT:**
  - `NULL` is a special state marker indicating that a value is **unknown, missing, or inapplicable**.
  - `NULL` is **NOT** numeric zero (`0`), **NOT** an empty string (`''`), and **NOT** a string of spaces (`'  '`).
  - Its behaviour must be uniform and **independent of column data type** (`INT`, `VARCHAR`, `DATE`, `DECIMAL`).
- **Internal Storage vs External Display:**
  - Internally, the DBMS sets a dedicated bit/marker in the row header to flag that the column is `NULL` while displaying `NULL` (or blank) to the user.
- **Tri-Valued Logic (3VL: `TRUE`, `FALSE`, `UNKNOWN / MAYBE`):**
  - Classic Boolean algebra has only two truth values (`TRUE` and `FALSE`). Introducing `NULL` forces the relational engine to operate under **Three-Valued Logic (3VL)**:
  - Any comparison with `NULL` using `=`, `!=`, `<`, or `>` (`SAL = NULL` or `NULL = NULL`) evaluates to **`UNKNOWN (MAYBE)`**, never `TRUE`! Hence SQL requires `IS NULL` and `IS NOT NULL`.

Predicate P	Predicate Q	P AND Q	P OR Q	NOT P
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

- **Entity Integrity Enforcement:** When a column is declared as a `PRIMARY KEY` (or `NOT NULL`), the DBMS systematically rejects any `INSERT` or `UPDATE` attempting to place a `NULL` in that attribute.

Rule 4: Dynamic Online Catalog Based on the Relational Model

Rule 4: *The data base description is represented at the logical level in the same way as ordinary data, so that authorized users can apply the same relational language to its interrogation as they apply to the regular data.*

- **Self-Describing Database (Metadata as Tables):**
  - A relational database maintains a **Data Dictionary / System Catalog** (containing metadata about all databases, tables, columns, data types, keys, views, indexes, constraints, and user privileges).
  - Instead of storing this metadata in proprietary binary files requiring a separate tool to inspect, Rule 4 mandates that the catalog itself is stored as **relational tables** online in memory/disk.
- **Practical MySQL Implementation (`INFORMATION_SCHEMA`):**
  - Authorised users and database administrators use standard SQL `SELECT` queries against `INFORMATION_SCHEMA` to inspect the schema dynamically at runtime:

```sql
-- Querying the dynamic online catalog in MySQL using standard SQL
SELECT TABLE_NAME, ENGINE, TABLE_ROWS
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'hr';

SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, COLUMN_KEY
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'hr'
  AND TABLE_NAME = 'employees';
```

Rule 5: The Comprehensive Data Sublanguage Rule

Rule 5: *A relational system may support several languages and various modes of terminal use (for example, the fill-in-the-blanks mode). However, there must be at least one language whose statements are expressible, per some well-defined syntax, as character strings and that is comprehensive in supporting all of the following items:* 1. *Data Definition* 2. *View Definition* 3. *Data Manipulation (interactive and by program)* 4. *Integrity Constraints* 5. *Authorization* 6. *Transaction Boundaries (`BEGIN`, `COMMIT`, and `ROLLBACK`)*

- **Historical Note (Alpha vs SQL):** Codd originally proposed his own predicate-calculus sublanguage called **Alpha**, while IBM's Donald Chamberlin and Raymond Boyce developed **SEQUEL / SQL**, which became the universal ANSI/ISO relational data sublanguage.
- **Why Character-String Expressibility Matters:** Because every SQL command is a plain character string (`"SELECT * FROM emp WHERE deptno = 10;"`), it can be typed interactively in a console OR dynamically constructed and embedded inside host programming languages (PHP, Python, Java, C#) and sent over network sockets.

Rule 6: The View Updating Rule

Rule 6: *All views that are theoretically updatable are also updatable by the system.*

- **What a View Is:**

  - A **View** is a virtual (derived) table defined by a stored `SELECT` query in the system catalog. It holds no persistent data of its own; rather, it dynamically projects/filters/joins underlying base tables whenever queried.
  - Views serve two primary purposes:
    1. **Simplifying Complex Queries / Summarisation:** Hiding complex joins behind a clean virtual table name.
    2. **Security and Access Control:** Restricting users to see only specific columns (hiding `salary`) or specific rows (`WHERE department_id = 30`).
- **The View Update Challenge:**
  - If a user executes `UPDATE`, `INSERT`, or `DELETE` against a View, the DBMS must translate that modification unambiguously back to the underlying base table(s).
  - **Theoretically Updatable vs Non-Updatable Views:** A simple view over a single base table that includes the base table's Primary Key is theoretically updatable. Conversely, a view containing `GROUP BY`, `AVG()`, `DISTINCT`, or set operators cannot be unambiguously inverted back to individual base rows.
  - Commercial RDBMSs struggle to support *all* theoretically updatable join views natively (Oracle provides **`INSTEAD OF` triggers** to let developers code the inverse base-table updates manually).

Rule 7: High-Level Insert, Update, and Delete (Set-at-a-Time Processing)

Rule 7: *The capability of handling a base relation or a derived relation as a single operand applies not only to the retrieval of data but also to the insertion, update and deletion of data.*

- **Set-at-a-Time vs Record-at-a-Time Processing:**
  - Pre-relational systems forced programmers to write procedural `FOR`/`WHILE` cursor loops that fetched, modified, and wrote one record at a time across the network.
  - Rule 7 states that `INSERT`, `UPDATE`, and `DELETE` must operate on **whole sets of tuples as a single operand** in one declarative statement:

```
-- Set-at-a-time UPDATE: updates 1 or 100,000 rows in a single relational
 ↪  operation
UPDATE employees
SET salary = salary * 1.05
WHERE department_id = 50;

-- Set-at-a-time INSERT from a derived relation
INSERT INTO high_earners (emp_id, ename, sal)
SELECT employee_id, last_name, salary
FROM employees
WHERE salary > 10000;
```

- Set-at-a-time operations allow the DBMS query optimiser to choose the best bulk execution strategy and minimise network round-trips.

Rule 8: Physical Data Independence

Rule 8: *Application programs and terminal activities remain logically unimpaired whenever any changes are made in either storage representations or access methods.*

- **Architectural Mapping:** Corresponds to the boundary between the **Internal (Physical) Level** and the **Conceptual (Logical) Level** in the ANSI/SPARC architecture.
- **Examples of Physical Changes That Must Not Break Applications:**
  - Creating, dropping, or changing an index from a B+Tree to a Hash index.
  - Moving tablespaces across physical hard drives or SSD volumes.
  - Changing page block sizes, compression algorithms, file sorting order, or storage engines.
- Because SQL queries reference only logical table and column names (never physical file paths or index names), all application code continues running without modification.

Rule 9: Logical Data Independence

Rule 9: *Application programs and terminal activities remain logically unimpaired when information-preserving changes of any kind that theoretically permit unimpairment are made to the base tables.*

- **Architectural Mapping:** Corresponds to the boundary between the **Conceptual (Logical) Level** and the **External (View) Level**.
- **Examples of Information-Preserving Logical Changes:**
  - **Splitting a table vertically (Normalization):** Suppose `EMPLOYEE(emp_id, name, dept_id, salary)` is split into two base tables `EMP_PUBLIC(emp_id, name, dept_id)` and `EMP_PAY(emp_id, salary)`. By creating a view `CREATE VIEW EMPLOYEE AS SELECT ... FROM EMP_PUBLIC NATURAL JOIN EMP_PAY`, legacy applications querying `EMPLOYEE` continue to work unimpaired!
  - **Splitting a table horizontally** across partitions or **adding new columns / tables** to the schema.
- Why Rule 9 is harder to achieve than Rule 8: Full logical data independence for write operations relies directly on **Rule 6 (View Updating)** so that `INSERT`/`UPDATE`/`DELETE` statements issued against the compatibility view propagate cleanly to the split base tables.

Rule 10: Integrity Independence

Rule 10: *Integrity constraints specific to a particular relational data base must be definable in the relational data sublanguage and storable in the catalog, not in the application programs.*

- **Two Minimum Mandatory Integrity Rules Specified by Codd:**
  1. **Entity Integrity:** No component of a **Primary Key** is allowed to have a `NULL` value.
  2. **Referential Integrity:** For every distinct non-null **Foreign Key** value in a relational database, there must exist a matching **Primary Key** value from the same domain in the referenced parent table.

- **Why Storing Constraints in the Catalog (Not Application Code) Is Critical:**
  - If business and integrity rules are coded only inside a Java/PHP frontend application, any new mobile app, batch script, or SQL console user connecting to the database can accidentally insert invalid data.
  - By declaring constraints (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, `CHECK`) in SQL DDL and storing them in the **Data Dictionary (Catalog)**:
    * Constraints can be modified dynamically as business rules evolve without recompiling application code.
    * Every application and user is centrally policed by the DBMS engine before any write is allowed to commit.

Rule 11: Distribution Independence

Rule 11: *A relational DBMS has distribution independence.*

- **Meaning (Single Logical Image Over a Distributed Database):**
  - Application programs and end-user queries must remain logically unimpaired whether data is stored on a single local server, **first distributed** across multiple geographic servers/nodes, or **later redistributed** across different sites.
  - To the user and application, a distributed database (whether **homogeneous**—running the same DBMS at every node—or **heterogeneous**—spanning Oracle, MySQL, and SQL Server nodes) must appear and behave like a **single local database**.
- **Six Key Engineering Challenges of Distributed RDBMSs:**
  1. **Replication:** Deciding the degree of replication across nodes and keeping replicas synchronised.
  2. **Location Transparency:** Locating data across the network without requiring users to specify server IP addresses or node names in their `SELECT` queries.
  3. **Fragmentation Transparency:** Supporting horizontal fragmentation (splitting rows across sites, e.g., European vs US customers) and vertical fragmentation (splitting columns across sites) and seamlessly reassembling them via joins/unions.
  4. **Global Naming:** Ensuring unique object naming across autonomous sites.
  5. **Distributed Transactions & Locking:** Coordinating Two-Phase Commit (2PC), distributed locks, and distributed deadlock detection across network links.
  6. **Catalog Placement:** Deciding where the system catalog resides (centralised on one node, fully replicated at every site, or dispersed with persistent location-independent identifiers).

Rule 12: The Non-Subversion Rule

Rule 12: *If a relational system has a low-level (single-record-at-a-time) language, that low-level language cannot be used to subvert or bypass the integrity rules and constraints expressed in the higher level relational language (multiple-records-at-a-time).*

- **Meaning:**

- Many DBMSs provide low-level C APIs, cursor interfaces, or high-speed bulk-loader utilities that operate one record at a time.
- Rule 12 mandates that **no low-level interface or utility may bypass the catalog's security and integrity rules** (Rule 10). All data entering the database through any path must obey the exact same Entity, Referential, and Domain integrity constraints.

Summary Table of Codd's 12 Rules

Rule #	Rule Name	Core Requirement Summary
Rule 0	**Foundation Rule**	System must manage databases *entirely* through its relational capabilities.
Rule 1	**Information Rule**	All data (and metadata) is represented *exclusively* as values in 2D tables (relations).
Rule 2	**Guaranteed Access Rule**	Every atomic value is logically addressable via `(Table Name, Primary Key Value, Column Name)`.
Rule 3	**Systematic Treatment of Nulls**	`NULL` (distinct from `0` or `''`) is supported uniformly across all data types using **3-Valued Logic (3VL)**.
Rule 4	**Dynamic Online Catalog**	The Data Dictionary (`INFORMATION_SCHEMA`) is stored as relational tables and queried with standard SQL.
Rule 5	**Comprehensive Data Sublanguage**	At least one character-string language (SQL) supports DDL, Views, DML, Integrity, Auth, and Transactions.
Rule 6	**View Updating Rule**	All views that are *theoretically* updatable must also be updatable by the DBMS.
Rule 7	**High-Level Insert, Update, Delete**	Set-at-a-time processing applies to `INSERT`, `UPDATE`, and `DELETE` as well as `SELECT`.
Rule 8	**Physical Data Independence**	Changes to physical storage, files, or indexes do not break application programs.

Rule #	Rule Name	Core Requirement Summary
Rule 9	**Logical Data Independence**	Information-preserving changes to base tables (e.g., splitting tables behind views) do not break applications.
Rule 10	**Integrity Independence**	Integrity constraints (Entity & Referential Integrity) are stored in the catalog, not in application code.
Rule 11	**Distribution Independence**	Distributed/fragmented/replicated data across multiple sites appears to applications as a single local DB.
Rule 12	**Non-Subversion Rule**	Low-level record-at-a-time interfaces cannot bypass catalog integrity rules defined in high-level SQL.

Week 7: Post-Relational (NoSQL) Databases, Data Architecture Patterns, and MongoDB

Introduction to Post-Relational (NoSQL) Databases

Defining NoSQL ("Not Only SQL")

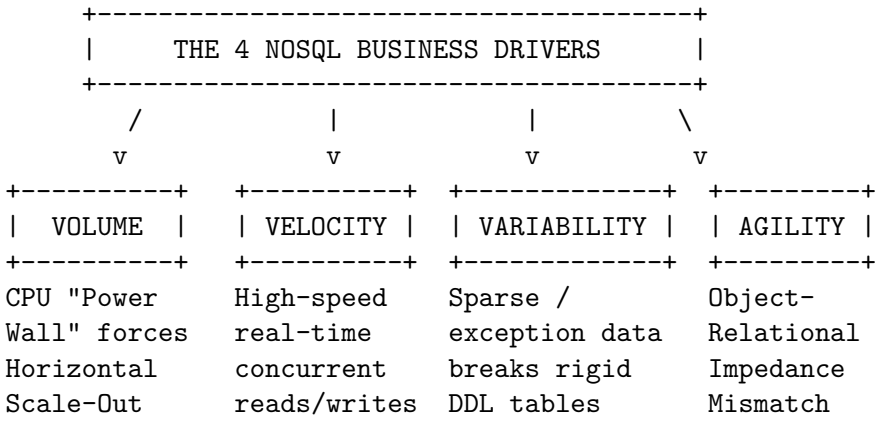
- **Origin of the Term:** First coined in 1998 by Carlo Strozzi for a lightweight open-source relational database that did not use SQL, and re-adopted in 2009 (by Johan Oskarsson and Eric Evans) to describe a new generation of **distributed, non-relational data stores**. Today, **NoSQL** is universally interpreted as **"Not Only SQL"**.
- **Formal Definitions:**
  - **Dan McCreary & Ann Kelly (*Making Sense of NoSQL*):** *"NoSQL is a set of concepts that allows the rapid and efficient processing of data sets with a focus on* ***performance, reliability, and agility****."*
  - **General Definition:** A NoSQL database provides a mechanism for the storage and retrieval of data that is modelled in means **other than the tabular relations** used in relational databases.

Core Characteristics of NoSQL Systems vs What NoSQL Is NOT

What NoSQL **IS** (Core Characteristics)	What NoSQL **IS NOT** (Common Misconceptions)
More than rows in tables: Stores data as Key-Value pairs, Wide-Column families, Graphs, or JSON/BSON/XML Documents.	**Not about rejecting the SQL language:** Many NoSQL systems provide SQL-like declarative query languages (e.g., Cassandra CQL).
Free of multi-table Joins: Co-locates related data to avoid expensive distributed join operations.	**Not "open-source only":** Includes both open-source (MongoDB, Cassandra, Redis) and commercial systems (MarkLogic, DynamoDB).
Schema-free / Flexible Schema: Allows storing and querying data without defining a rigid upfront ER schema (`CREATE TABLE`).	**Not only for "Big Data":** Frequently chosen for small-to-medium apps purely for schema agility and developer speed.
Runs on many processors (Shared-Nothing Architecture): Distributes data across clusters of cheap commodity nodes, each with its own independent RAM and local disk.	**Not exclusively about Cloud Computing:** Runs equally well in on-premise corporate data centres or private clusters.
Supports Linear Horizontal Scalability (Scale-Out): Doubling the number of commodity nodes roughly doubles read/write capacity.	**Not about clever RAM/SSD hardware tricks** and **not an elite niche group of products**.

The Four NoSQL Business Drivers: Why Single-Node RDBMSs Cracked

Four fundamental engineering and business forces drove the emergence of NoSQL architectures: **Volume**, **Velocity**, **Variability**, and **Agility**.



Volume (The CPU "Power Wall" and Vertical vs Horizontal Scaling)

- For decades, organisations scaled relational databases **vertically (Scale-Up)**—buying a larger, more expensive mainframe or SMP server with a faster single CPU clock speed.
- Around the mid-2000s to 2018, processor physics hit the **"Power Wall"** (thermal dissipation and instruction-level parallelism limits flattened single-core clock speeds). Instead of faster

single cores, hardware shifted to **multi-core processors and clusters of cheap commodity servers**.
- Traditional RDBMSs (designed around single-node shared-memory locking and cross-table joins) struggle to scale across dozens of separate servers. NoSQL databases were engineered from day one to **scale horizontally (Scale-Out)** via **sharding** and parallel processing across shared-nothing clusters.

Velocity (High-Frequency Real-Time Reads and Writes)

- Even when dataset size is moderate, modern web storefronts, IoT telemetry, and gaming backends face **extreme burst write and read velocity**.
- A single-processor RDBMS enforces strict ACID locking, undo/redo logging, and index maintenance on every write, causing severe queuing bottlenecks under heavy concurrent write bursts when thousands of users hit the database simultaneously.

Variability (Sparse and Polymorphic Exception Data)

- Companies that want to capture and store custom **exception data** or rapidly evolving attributes run into trouble with rigid RDBMS schemas.
- In a relational database, storing a one-off attribute that applies to only 1% of rows requires running an expensive `ALTER TABLE ADD COLUMN` DDL migration across millions of rows (filling the other 99% of rows with `NULL`s). Schema-free NoSQL stores accept varying attributes per record effortlessly.

Agility (Overcoming Object-Relational Impedance Mismatch)

- In modern Agile development, applications manipulate rich **nested in-memory objects** (lists, dictionaries, sub-objects).
- Mapping a single nested application object into a normalized RDBMS requires shredding it across 5 or 6 relational tables on `INSERT`/`UPDATE` and reassembling it with complex multi-table `JOIN`s on `SELECT` (**Object-Relational Impedance Mismatch**).
- Storing the object directly as a single JSON/BSON document accelerates development cycles and schema iteration.

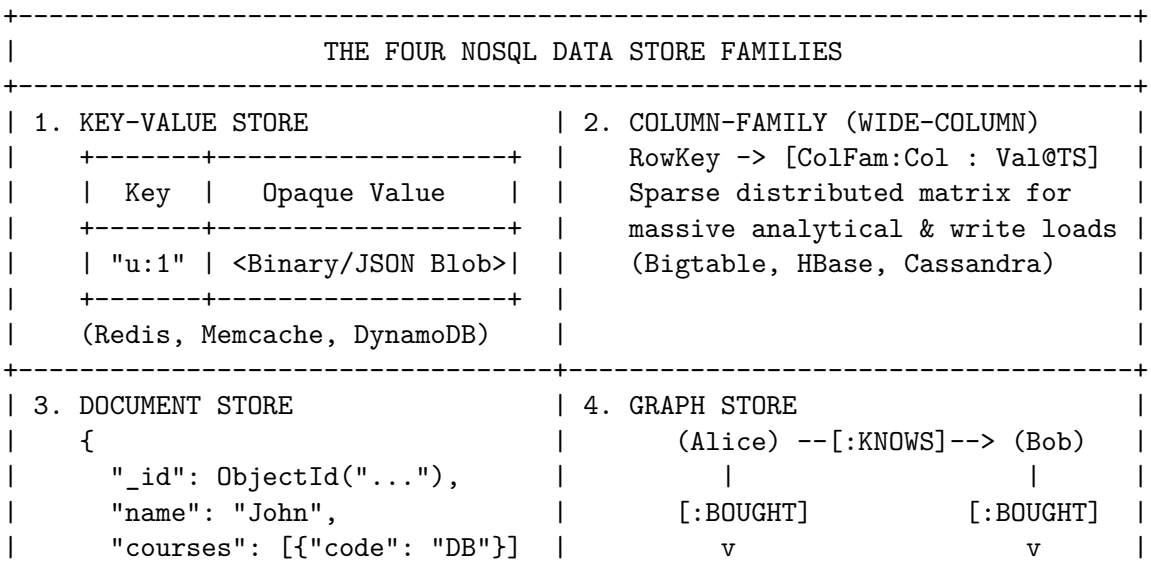
Five Foundational Industry Case Studies That Pioneered NoSQL

Pioneer & System	Problem Faced	Architectural Innovation
LiveJournal (`Memcache`)	Millions of page views overwhelmed backend RDBMS read capacity.	Hashed query results into a distributed **in-memory Key-Value cache** in RAM, drastically cutting RDBMS read traffic.

Pioneer & System	Problem Faced	Architectural Innovation
Google (`MapReduce`)	Indexing billions of web pages across the internet was impossible on a single supercomputer.	Divided massive batch computations into parallel **`Map` (filter/extract)** and **`Reduce` (aggregate)** tasks across thousands of commodity machines.
Google (`Bigtable`)	Needed to store petabytes of web crawl results (billions of URLs × millions of dynamic attributes).	Invented the **Sparse Matrix / Wide-Column Family store** over Google File System (GFS) with zero upfront ER schema modelling.
Amazon (`Dynamo`)	Any downtime in the e-commerce shopping cart directly cost millions in lost sales.	Built a peer-to-peer **Key-Value store** prioritizing **24/7 Always-On Availability (`AP`)** and eventual consistency so users can *always* add items to their cart.
MarkLogic	Enterprise publishing and intelligence agencies needed to search millions of complex hierarchical XML documents.	Distributed XML **Document Store** indexing and querying nested documents across local disks on commodity server clusters.

The Four NoSQL Data Store Families

NoSQL databases are categorised into **four primary architectural families** based on their underlying data model:



```
|    }                                |              (Database Book)        |
|    (MongoDB, CouchDB, MarkLogic)  |    (Neo4j, AllegroGraph, Titan)     |
+-----------------------------------+-------------------------------------+
```

NoSQL Family	Core Data Model & Mechanics	Typical Use Cases	Representative Products
1. Key-Value Store	Simplest NoSQL system: a distributed hash table mapping a unique **Key** to an arbitrary **Opaque Value (BLOB)** (`put`, `get`, `delete`).	Session caches, shopping carts, user profiles, image/file stores, high-speed lookup tables.	**Redis**, **Memcache**, **Berkeley DB**, **Riak**, **Amazon DynamoDB**.
2. Column-Family (Wide-Column) Store	**Sparse Matrix** system indexing data by **Row Key**, **Column Family / Qualifier**, and **Timestamp**. Rows do not need to share the same columns.	Web crawler storage, time-series telemetry, massive write-heavy logs, big data analytics.	**Google Bigtable**, **Apache HBase**, **Apache Cassandra**, **Hypertable**, **Apache Accumulo**.
3. Document Store	Stores self-describing, hierarchical trees of **semi-structured documents** (**JSON**, **BSON**, **XML**) inside collections; supports indexing inside document fields.	Content Management Systems (CMS), e-commerce product catalogs, CRM, high-variability data, real-time analytics.	**MongoDB**, **Apache CouchDB**, **Couchbase**, **MarkLogic**.
4. Graph Store	Based on mathematical **Graph Theory**: stores **Nodes** (entities), directed **Edges** (relationships), and key-value **Properties** on both nodes and edges.	Social networks, fraud rings, recommendation engines, network/IT topology, routing.	**Neo4j** (Cypher), **AllegroGraph**, **InfiniteGraph**, **Titan / JanusGraph**.

Data Architecture Patterns: Row-Store RDBMS vs Key-Value Store Deep Dive

Evolution of Enterprise Data Architecture (`OLTP`, `OLAP`, and Polyglot Persistence)

1. **1970s–1990s (Siloed Departmental Systems):** Separate isolated databases for `HR`, `Finance`, `Inventory`, and `Sales`, causing massive data duplication.

2. **1990s–2000s (ERP + Data Warehouse / BI):** Operational **ERP (Enterprise Resource Planning)** systems consolidated daily transactional workloads (**OLTP — Online Transaction Processing**), while nightly ETL jobs fed historical data into **Data Warehouses** for reporting and business intelligence (**OLAP — Online Analytical Processing**), alongside separate unstructured Document repositories.
3. **Modern Era (Polyglot Persistence):** Enterprises combine **ERP (Relational OLTP) + BI / Data Warehouses (OLAP) + NoSQL Document / Key-Value / Graph / Columnar Stores**, selecting the right data architecture pattern for each workload.

Dimension	**OLTP (Online Transaction Processing)**	**OLAP (Online Analytical Processing)**
Primary Purpose	Run day-to-day real-time business operations (orders, banking, inventory updates).	Historical analysis, trend forecasting, executive dashboards, and ad-hoc BI reporting.
Workload Pattern	High volume of short, concurrent `INSERT`, `UPDATE`, `DELETE`, and point `SELECT` transactions.	Complex read-heavy aggregations (`SUM`, `AVG`, `COUNT`) scanning millions of historical rows.
Schema Design	Highly normalized (`3NF`) Row-Store tables to prevent update anomalies.	Denormalized Star / Snowflake schemas or Columnar stores optimised for fast scans.

Evaluation of the RDBMS Row-Store Architecture Pattern

In a traditional RDBMS **Row-Store Pattern**, all columns belonging to a single tuple are stored **contiguously together on disk**, rows are inserted/deleted as a unit, and column types are fixed at `CREATE TABLE` time.

RDBMS Design Pattern Feature	**Pros (Strengths)**	**Cons (Limitations at Scale)**
Joins Between Tables	Easy to create logical views combining multiple normalized tables.	**All joined tables must reside on the same server**; distributed joins across network nodes are slow and difficult to scale horizontally.
ACID Transactions	Clear, reliable `BEGIN` / `COMMIT` / `ROLLBACK` boundaries for multi-row updates.	Readers and writers can contend under strict isolation levels; distributed locking adds heavy latency.
Fixed DDL & Typed Columns	Enforces strict domain rules (`DATE`, `INT`, `DECIMAL`) and enables fast B-Tree range indexes.	Rigid when handling sparse/exception attributes; `ALTER TABLE` on massive tables is disruptive.

RDBMS Design Pattern Feature	**Pros (Strengths)**	**Cons (Limitations at Scale)**
Fine-Grained Security	Mature row- and column-level security via SQL Views and `GRANT/REVOKE`.	Can become complex to administer across hundreds of dynamic application roles.
Document Integration	Can store raw text in `BLOB/TEXT/JSON` columns.	Poor native architecture for deeply nested hierarchical documents compared to native Document Stores.

Deep Dive into the Key-Value Store Architecture Pattern (`PRDB 3`)

Conceptual Model: The Dictionary Analogy A **Key-Value Store** operates like a giant distributed dictionary or associative array (`Map<Key, Value>`):

Application Scenario	Example **Key** (`xs:string`)	Example **Value** (`item()` / Opaque BLOB)
English Dictionary	Word `"amphora"`	Definition text, pronunciation audio, and image of a Greek vase
Web Image / Asset Cache	Path `"/images/logo.png"`	Binary PNG image bytes
Web Page Cache	URL `"https://example.com/home"`	Rendered HTML page string
Document / File Store	MD5 Hash `"9e107d9d372bb682..."`	Complete PDF or XML document
Database Query Cache	SQL String `"SELECT * FROM emp..."`	Serialised JSON/XML result set

The Three Core API Operations of a Key-Value Store Every Key-Value store exposes a minimal, ultra-fast three-operation interface:

1. **`put($key as xs:string, $value as item()):`**
   - Creates a new key-value entry in the store, OR **overwrites/updates** the existing value if `$key` is already present.
2. **`get($key as xs:string) as item():`**
   - Looks up `$key` via a direct hash computation ($O(1)$ lookup) and returns the associated `$value` (or misses/errors if `$key` does not exist).
3. **`delete($key as xs:string):`**
   - Removes `$key` and its associated `$value` from the keyspace.

Two Fundamental Rules of a Pure Key-Value Store

1. **Rule 1 — Distinct Keys:** Every key within a keyspace/bucket must be **strictly unique** (you can never have two entries with the same key).

2. **Rule 2 — No Queries on Values:** To the Key-Value store engine, the `Value` compartment is a completely **opaque BLOB ("black box")**. You **cannot** write a `WHERE` filter or index on contents inside the `Value`—all lookups must be performed strictly by `Key`!

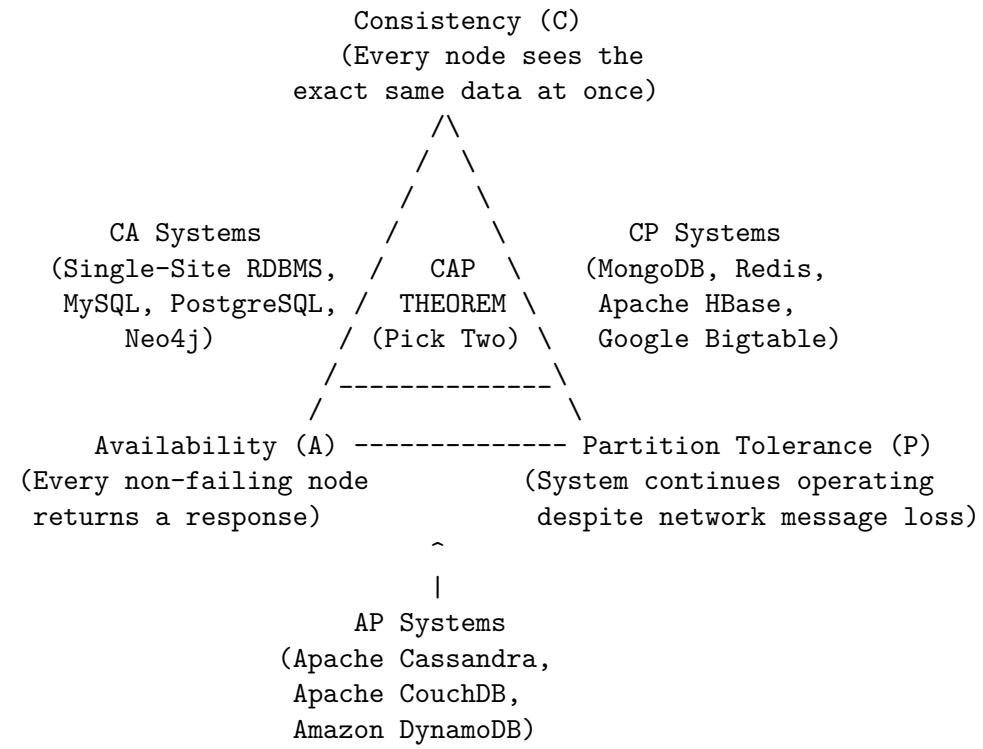
Precision Service Level Agreements (SLAs) and Portability

- Because the engine only executes `put`, `get`, and `delete` by hash key, architects can tune **Precision Service Levels**:
  - Guaranteed maximum latency (e.g., ≤ 5 ms per `get` or `put`).
  - Target throughput (reads and writes per second).
  - Configurable **Replication Factor** across geographic data centres.
  - Tunable consistency (trading strict immediate consistency for eventual consistency).
- **High Portability:** Because the application only calls `put(k, v)`, `get(k)`, and `delete(k)`, swapping one underlying Key-Value product for another requires minimal code changes.

Distributed Database Theory: The CAP Theorem and ACID vs BASE

Brewer's CAP Theorem

Formulated by **Eric Brewer (2000)** and proved by Gilbert and Lynch (2002), the **CAP Theorem** states that any distributed shared-data system can simultaneously guarantee **at most TWO out of THREE** desirable properties when a network partition occurs:



1. **Consistency (*C* — Linearizability):** Every read on any node in the cluster is guaranteed to return the **most recent write** (all nodes see the exact same data at the exact same instant).

2. **Availability (A):** Every request received by a non-failing node is guaranteed to result in a non-error response (without guaranteeing that it contains the latest write from a partitioned peer).
3. **Partition Tolerance (P):** The cluster continues to operate despite arbitrary network partitions (dropped or delayed messages between nodes in different racks or data centres).

- **Why Distributed Systems Must Choose Between `CP` and `AP`:**
  - In any real multi-node network, **network partitions (P) are inevitable** (cables get cut, switches fail).
  - When a partition splits Node A and Node B:
    * A `CP` **System** (**MongoDB**, **HBase**, **Redis**) refuses writes/reads on the minority partition to preserve **Consistency (C)**, sacrificing 100% availability.
    * An `AP` **System** (**Apache Cassandra**, **CouchDB**, **DynamoDB**) allows both sides of the partition to keep accepting reads and writes to preserve **Availability (A)**, resolving conflicts later once the partition heals (**Eventual Consistency**).

ACID vs BASE Consistency Models

Property Dimension	**ACID (Traditional Relational DBMS)**	**BASE (Distributed NoSQL Systems)**
Acronym Breakdown	**A**tomicity, **C**onsistency, **I**solation, **D**urability	**B**asically **A**vailable, **S**oft state, **E**ventually consistent
Primary Design Goal	Strict immediate data correctness; zero stale reads allowed.	High availability, partition resilience, and low-latency horizontal scale.
State After Write	Immediately consistent across the database upon `COMMIT`.	**Soft State:** Replicas may temporarily hold different versions while asynchronous replication is in flight.
Convergence Guarantee	Immediate (synchronous locking / 2PC).	**Eventual Consistency:** If no new writes arrive for a period of time, all replicas eventually converge to the exact same state.

Distributed Replication Topologies: Master-Slave vs Peer-to-Peer (Masterless)

1. **Master-Slave (Primary-Secondary) Replication (Used by MongoDB Replica Sets & Redis):**
   - One node is elected as the **Primary (Master)** node; all other nodes in the replica set act as **Secondaries (Slaves)**.
   - **All Writes (`INSERT`/`UPDATE`/`DELETE`) go to the Primary node**, which logs operations to its **oplog** and asynchronously replicates them to Secondaries.
   - **Reads** can be served by the Primary or scaled out across Secondaries.

- **Automatic Failover via Heartbeats:** Nodes exchange periodic **heartbeat** pings every few seconds. If the Primary node goes down, the remaining Secondaries hold a majority election and automatically promote a Secondary to become the new Primary!

2. **Master-Master / Peer-to-Peer Ring Topology (Used by Apache Cassandra):**
   - **Every node is identical**—there is **no single Master node** and **no single point of failure (SPOF)**.
   - Any client can send a Read or Write request to **any node** in the cluster (which acts as the coordinator for that request) using consistent hashing around a ring.
   - Supports tunable **Quorum Consistency** ($R + W > N$, where N is the replication factor, W is the write quorum, and R is the read quorum).

MongoDB Architecture, BSON, and CRUD Operations

Relational RDBMS vs MongoDB Terminology Mapping

Relational DBMS (MySQL / Oracle)	MongoDB (Document Store)	Technical Notes
Database	**Database**	Logical container for collections (`use schoolDB`).
Table / Relation	**Collection**	Dynamic group of documents (`db.student`); does not enforce a rigid column schema by default.
Row / Tuple	**Document (JSON / BSON)**	Self-contained hierarchical record up to **16 MB**.
Column / Attribute	**Field**	Key-value pair inside a document (can hold scalars, arrays, or nested sub-documents).
Primary Key (`PK`)	**`_id` Field (`ObjectId`)**	Mandatory, **immutable** primary key indexed automatically on every document; auto-generated as a 12-byte `ObjectId` if omitted.
Index	**Index**	B-Tree indexes supported on top-level fields, nested sub-fields, arrays (multikey), text, and geospatial coordinates.
Table `JOIN`	**Embedded Documents**, **Manual References / `DBRef`**, or **`$lookup`**	Related data is either nested inside the parent document or linked via `_id` references.

BSON (Binary JSON), the 16 MB Document Limit, and GridFS

- **JSON vs BSON:**
  - While users write queries and documents in human-readable **JSON (JavaScript Object Notation)**, MongoDB stores documents on disk and transmits them over the network in **BSON (Binary JSON)**.
  - **Advantages of BSON:**
    1. **Fast Binary Traversal:** Encodes length prefixes on strings and sub-objects so the WiredTiger engine can skip directly to target fields without parsing every character.
    2. **Rich Native Data Types:** Supports types missing from plain JSON, including `ObjectId` (12-byte unique timestamp+machine+counter ID), `Date` (`ISODate`), 32-bit `Int32`, 64-bit `Int64` (`Long`), `Decimal128`, raw `BinData`, and regular expressions.
- **16 MB Maximum Document Size & `GridFS`:**
  - Every single BSON document is capped at **16 MB** to prevent single gigantic documents from monopolising RAM or network bandwidth.
  - To store large files exceeding 16 MB (such as videos, audio recordings, or large PDFs), MongoDB provides **`GridFS`**, which automatically splits the file into **255 KB chunks** stored across two collections (`fs.files` for metadata and `fs.chunks` for binary chunks).

Essential MongoDB Shell (`mongosh`) CRUD Operations

Creating a Database, Collection, and Inserting Documents (`insertOne`, `insertMany`)

```
// Switch to (or create) database and collection
use universityDB;
db.createCollection("student");

// Insert a single hierarchical document with nested object and array
db.student.insertOne({
  name: { firstName: "John", lastName: "Doe" },
  dateOfBirth: "10/05/2001",
  college: "ABC College",
  courses: [
    { courseName: "Advanced Databases", creditHrs: 5 },
    { courseName: "Cybersecurity", creditHrs: 3 }
  ]
});

// Insert multiple documents at once
db.student.insertMany([
  {
    name: { firstName: "Jane", lastName: "Smith" },
    dateOfBirth: "22/08/2000",
    college: "ABC College",
    courses: [{ courseName: "Cloud Computing", creditHrs: 5 }]
  },
  {
    name: { firstName: "Alice", lastName: "Brown" },
```

```
    dateOfBirth: "14/01/2002",
    college: "XYZ Institute",
    courses: [{ courseName: "Data Mining", creditHrs: 3 }]
  }
]);
```

Querying Documents (`find` with Dot-Notation Filter and Projection)

```
// Find students enrolled in any 3-credit-hour course;
// Project only the 'name' field (1) and suppress '_id' (0)
db.student.find(
  { "courses.creditHrs": 3 },
  { "name": 1, "_id": 0 }
);
```

Updating Documents (`updateOne`, `updateMany` with `$set`)

```
// Update John's dateOfBirth using the $set operator
db.student.updateOne(
  { "name.firstName": "John" },
  { $set: { dateOfBirth: "12/12/2002" } }
);

// Update multiple documents matching a filter
db.student.updateMany(
  { college: "ABC College" },
  { $set: { accredited: true } }
);
```

Deleting Documents (`deleteOne`, `deleteMany`)

```
// Delete all student documents from ABC College
db.student.deleteMany({ college: "ABC College" });
```

Horizontal Sharding Strategies in MongoDB

To scale horizontally across a cluster of shard servers, MongoDB partitions a collection based on a chosen **Shard Key** using one of three strategies:

1. **Ranged Sharding:** Partitions data into contiguous ranges of the shard key (e.g., `zipcode` `10000..19999` on Shard A, `20000..29999` on Shard B). Efficient for **range queries**, but a monotonically increasing key (like a timestamp or `ObjectId`) can create a "hot shard" receiving all inserts.
2. **Hashed Sharding:** Computes a cryptographic hash of the shard key to distribute documents **evenly and randomly** across all shards. Ideal for uniform write distribution, though range queries must scatter-gather across all shards.
3. **Zoned Sharding:** Associates shard key ranges with specific physical **Zones / Data Centres** (e.g., pinning EU customer documents to European servers for GDPR data residency compliance).

Schema Design in MongoDB: Normalization (Referencing) vs Denormalization (Embedding)

One of the most important architectural decisions in MongoDB (explored in detail in the Moodle Week 7 research paper *A Study of Normalization and Embedding in MongoDB*) is deciding when to **Embed (Denormalize)** related entities inside a single document versus when to **Reference (Normalize)** them across separate collections.

Side-by-Side Structure: Embedded vs Referenced Models

```
// APPROACH A: EMBEDDING (Denormalized Single Document)
// Best for 1:1 and bounded 1:Few relationships read together
{
  _id: ObjectId("654a01..."),
  empName: "Sarah Connor",
  address: { street: "12 Main St", city: "Galway" }, // 1:1 Embedded Object
  children: [                                        // 1:Few Embedded Array
    { name: "John", birthYear: 2015 },
    { name: "Ella", birthYear: 2018 }
  ]
}

// APPROACH B: REFERENCING (Normalized Separate Collections)
// Best for 1:Many (unbounded) and M:N relationships
// Collection 1: publisher
{ _id: "OReilly", name: "O'Reilly Media", location: "CA" }

// Collection 2: book (stores Foreign-Key-style reference to publisher._id)
{ _id: 9001, title: "Learning MongoDB", publisher_id: "OReilly" }
```

Empirical Performance and Trade-Off Comparison

In the benchmark study comparing **Unnormalized / Embedded (`UNF` / `1NF`)** schemas against **Normalized Referenced (`2NF` / `3NF`)** schemas across **103,570 records** in MongoDB:

Evaluation Criterion	**Embedding (Denormalization)**	**Referencing (Normalization)**
Read Query Performance	**Fast & Deterministic (1–2 s):** All related data sits contiguously in one document on disk (**Data Locality**), retrieved in a single disk seek without joins.	**Slower on Multi-Collection Queries:** Requires either multiple application-level round-trips or expensive **`$lookup` aggregation pipeline joins** (which stalled/timed out on complex 4-collection joins in `UNF`/`1NF` cross-collection tests).

Evaluation Criterion	**Embedding (Denormalization)**	**Referencing (Normalization)**
Atomicity of Writes	Single-document updates are **natively atomic** without needing multi-document locks.	Updating multiple collections requires multi-document ACID transactions (introduced in MongoDB 4.0+) with higher overhead.
Data Duplication & Anomalies	**High Duplication if Shared (`M:N`):** If the same `Department` or `Publisher` details are embedded inside 50,000 documents, updating the publisher's address requires updating all 50,000 documents.	**Zero Duplication:** Shared entity attributes are stored once in their own collection and updated in one place.
Document Size Growth	**Risk of Exceeding the 16 MB BSON Limit** if an embedded array grows without bound (e.g., millions of sensor logs or comments appended to one document).	Safe for unbounded growth because each child item is its own independent document in a separate collection.

Four Golden Rules for MongoDB Schema Modelling

1. **Favour Embedding for `1:1` and Bounded `1:Few` Relationships** where the child objects are always retrieved together with the parent and never need to be queried independently.
2. **Favour Referencing for `1:Many` (Unbounded / High-Cardinality) and `M:N` Relationships** (or when an embedded array could grow continuously toward the **16 MB** limit).
3. **Avoid Multi-Collection `$lookup` Joins on Hot Paths:** Design document boundaries around the application's most frequent read access patterns.
4. **Array Indexing Caveat:** Indexing large embedded arrays (Multikey Indexes) increases index size and write amplification; keep embedded arrays bounded.

Comprehensive Comparison Across Relational and NoSQL DBMS Families

The following synthesis table compares **MySQL (`InnoDB`)** against the four flagship NoSQL databases studied in the Week 7 literature (**MongoDB**, **Apache Cassandra**, **Redis**, and **Neo4j**):

Feature / Dimension	**MySQL (`InnoDB`)**	**MongoDB**	**Apache Cassandra**	**Redis**	**Neo4j**
Database Category	Relational DBMS (Row Store)	Document Store (BSON)	Column-Family / Wide-Column Store	In-Memory Key-Value Store	Graph Store (Labeled Property Graph)

Feature / Dimension	**MySQL (`InnoDB`)**	**MongoDB**	**Apache Cassandra**	**Redis**	**Neo4j**
CAP Classification	**CA** (Single-node / Cluster ACID)	**CP** (Consistent & Partition-Tolerant)	**AP** (Available & Partition-Tolerant)	**CP**	**CA**
Consistency Model	Strict **ACID**	Tunable (**ACID** single/multi-doc or BASE)	Tunable **BASE** (Eventual / Quorum)	**BASE** / Optimistic (`WATCH`/`MULTI`)	Strict **ACID**
Query Language	**SQL**	**MQL / `mongosh`** (JavaScript API)	**CQL** (Cassandra Query Language)	Key-Value Commands (`GET`, `SET`, `HGET`)	**Cypher** Pattern-Matching Language
Replication Model	Master-Slave / Group Replication	**Master-Slave Replica Sets** (Auto-failover via heartbeats)	**Masterless Peer-to-Peer Ring** (No SPOF)	**Master-Slave** (Redis Sentinel / Cluster)	Causal Clustering (Raft consensus)
Horizontal Partitioning	Manual Sharding / MySQL Cluster	**Automatic Sharding** (Ranged, Hashed, Zoned)	**Automatic Consistent Hashing** across ring	**Hash Slot Sharding** (16,384 slots)	Fabric / Graph Sharding
Read / Write Sweet Spot	Complex multi-table joins & strict transactional integrity.	Fast single-document reads/writes & flexible nested schemas.	**Extreme write velocity** & linear scale across data centres; slow on `DELETE` (uses **Tombstones**).	**Sub-millisecond RAM latency** for caching, pub/sub, and session state (persists via **RDB snapshots** & **AOF logs**).	Deep multi-hop **relationship traversals** ($O(1)$ index-free adjacency pointers vs slow SQL `JOIN`s).

Feature / Dimension	**MySQL (`InnoDB`)**	**MongoDB**	**Apache Cassandra**	**Redis**	**Neo4j**
Key Security Considerations	Mature SQL `GRANT/REVOKE` RBAC, Views, SSL/TLS, and prepared statements against SQLi.	Must explicitly enable **RBAC authentication**, TLS, and encryption at rest; sanitize JSON inputs against **NoSQL Injection** (`$ne`, `$gt`).	Enable password authentication, client-to-node TLS, and CQL role permissions.	Designed to run inside trusted private VPCs; bind to localhost/VPC and enable ACL passwords + TLS.	Role-Based Access Control (RBAC), property-level security rules, and Bolt TLS encryption.